五级流水线 MIPS 处理器设计实验报告

“节点放大器”汇编程序的 Verilog 实现与 FPGA 验证

姓名：吴泓锦　**学号：**2024010654　**班级：**无 48　**日期：**2026 年 8 月 5 日

目录

1　实验目的 **3**

2　设计方案 **3**

2.1　大作业要求概述 3

2.2　处理器总体结构与框图 4

2.3　原理说明 5

3　算法指令 **7**

3.1　题目要求与总体设计 7

3.2　输入格式与 RAM 布局 7

3.3　链式邻接表构建 8

3.4　递归 DFS 与动态规划 9

3.5　UART 输入输出的裸机实现 11

3.6　结果保存与软件七段译码 12

4　工程文件清单与关键代码 **13**

4.1　工程目录结构 13

4.2　rtl 文件夹 13

4.3　sim 文件夹 20

4.4　constraints 文件夹 21

4.5　asm 文件夹 22

4.6　data 文件夹 23

5 仿真结果与分析 **24**
5.1 仿真环境与验证方案 24
5.2 UART 外设仿真：前仿 1 24
5.3 完整 SoC RTL 功能仿真：前仿 2 26
5.4 实现后时序仿真 27

6 综合情况 **30**
6.1 综合与实现结果 30
6.2 资源占用情况 30
6.3 时序性能与最高工作频率 31
6.4 CPI 及每秒执行指令数 32

7 硬件调试情况 **35**

8 思想体会 **35**

A 优化设计说明 **36**
A.1 优化目标与总体方法 36
A.2 初始时序问题与关键路径分析 37
A.3 关键 RTL 优化措施 37
A.4 优化后的时序结果 41
A.5 补充说明 42

B 对 CPI 计算结果的印证 **43**

1 实验目的

本实验以“节点放大器”算法为应用任务，设计并实现一套能够在 FPGA 上运行裸机 MIPS 程序的五级流水线处理器系统。主要目的如下：

1. 复习理论课 MIPS 五级流水线处理器的数据通路、控制通路和流水线寄存器的设计方法，理解 IF、ID、EX、MEM、WB 各阶段的功能。
2. 分析并处理流水线中的数据冒险和控制冒险，实现 forwarding、load-use stall 以及分支、跳转指令的 flush 机制。
3. 将节点放大器汇编程序改写为适合 FPGA 裸机运行的形式，并通过存储器映射方式访问数据 RAM、七段数码管和 UART 外设，实现软件译码及串口输入输出。
4. 强化对 Vivado 的使用技能，完成处理器工程的仿真、综合、时序分析与上板验证，并通过 CPI、最高工作频率和资源利用率评价处理器的性能。

2 设计方案

2.1 大作业要求概述

本实验要求用 Verilog 实现五级 MIPS 流水线的 CPU 处理器，在 CPU 上执行“节点放大器”汇编程序，要求如下：

1. 设计 IF、ID、EX、MEM、WB 五级流水线处理器，实现完整的数据 forwarding；对 load-use 冒险 stall 一个周期并插入 nop；分支指令在 EX 阶段判定，分支成立时取消 IF、ID 阶段的错误指令；跳转指令在 ID 阶段处理，并取消 IF 阶段指令。
2. 支持 `lw`、`sw`、`lui`、算术逻辑、移位和比较等基本指令，并实现 `beq`、`bne`、`blez`、`bgtz`、`bltz` 以及 `j`、`jal`、`jr`、`jalr` 等分支跳转指令；具有可扩展性，可根据程序需要继续扩展其他 MIPS 指令。
3. 采用指令与数据分离的哈佛结构，将低地址空间映射为数据 RAM，将 `0x40000010` 映射为七段数码管外设。测试数据采用理论课规定的 32 位整数格式，计算结果须以十六进制显示，且七段码必须由汇编程序从 RAM 查表获得（软件译码），不得使用硬件译码器。
4. 在完成 RAM 初始化输入和数码管显示的基础功能上，实现 UART 提高要求：通过 `0x40000018`、`0x4000001C` 和 `0x40000020` 完成串口发送、接收及状态查询，以统一的 32 位整数格式输入测试数据并输出结果，同时保留数码管显示功能。系统还需能够在更换测试数据后重新综合上板，并通过仿真、综合、时序分析和硬件测试验证功能与性能。

2.2 处理器总体结构与框图

本工程采用哈佛架构，指令存储空间与数据存储空间独立。IF 阶段通过专用的指令地址和指令数据通路访问 Instruction ROM；MEM 阶段则通过另一组数据地址、读写控制和读写数据通路访问 Data RAM 或存储器映射外设。两类存储器具有独立的地址接口，不共用存储阵列和访问总线，因此取指与数据访问可在同时钟周期内并行进行。

处理器核内部按照 IF、ID、EX、MEM、WB 五个阶段组织，相邻阶段通过流水线寄存器传递数据和控制信号。Control、Hazard Unit 和 Forwarding Unit 共同处理指令译码、数据相关、stall 和 flush；WB 阶段将 ALU 结果、RAM 读出数据或链接地址写回寄存器堆。MEM 阶段产生的数据访问请求由 `AmplifierSoC` 完成地址译码，并分别选择 Data RAM、七段数码管或 UART。

流水线数据通路大体与理论课课件（图 1）一致，系统总体结构如图 2 所示。

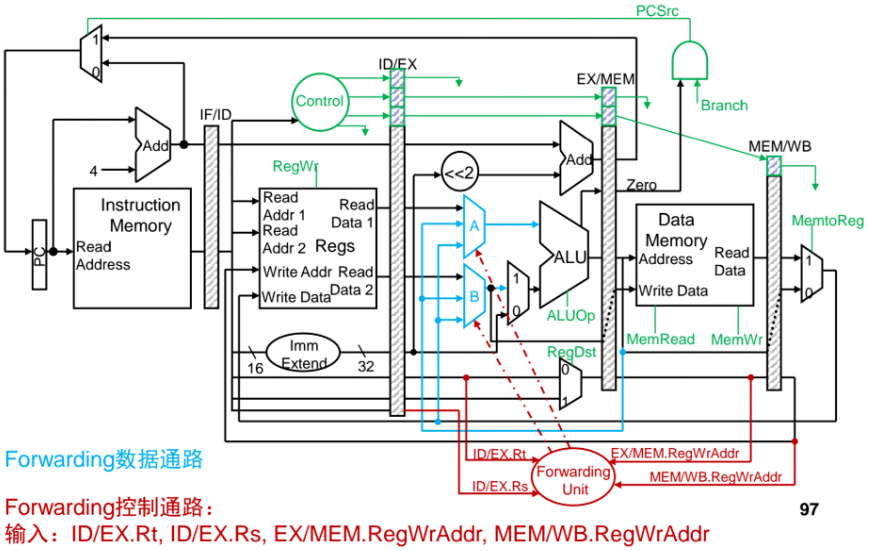


图 1：流水线数据通路（理论课课件）

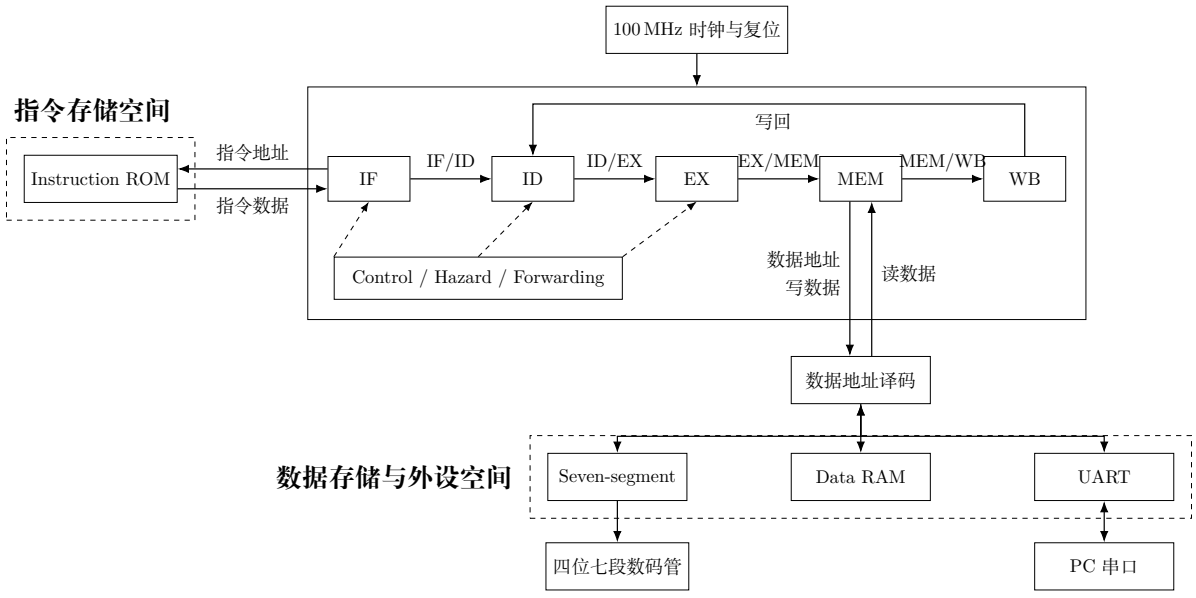


图 2：总体结构框图

2.3 原理说明

采用理论课所讲的 IF、ID、EX、MEM、WB 五级流水线结构，设置四组级间寄存器。这里不再叙述基本知识，主要说明工程的具体实现，以及存储和外设的组织方式。

2.3.1 流水线的 RTL 实现

在 `PipelineCPU` 中，PC 和四组级间寄存器均定义为时序状态，各自在时钟上升沿通过非阻塞赋值更新；指令译码、立即数扩展、ALU 运算、分支判断、转发选择和写回选择则采用组合逻辑实现。

ID 级生成的控制信号与指令数据一起存入 ID/EX，并继续传递到后续级间寄存器。其中，ALU 操作及操作数选择信号在 EX 级使用，访存控制信号传递至 MEM 级，寄存器写使能和写回来源选择则传递至 WB 级。写回数据可以来自 ALU 结果、Data RAM 读数据或 `PC+4`；目的寄存器可以选择 rt、rd 或 31 号寄存器，从而支持 `jal` 和 `jalr`。

ControlUnit 对指令码进行组合译码，并额外生成 `uses_rs`、`uses_rt`、分支类型和跳转类型等信息。寄存器堆采用双端口组合读、时钟上升沿写，并设置 WB 到 ID 的同周期旁路。节点放大器程序不依赖乘法，因此通过 `ENABLE_MUL=0` 关闭了组合乘法通路。

流水线控制通过改变 PC 和级间寄存器的更新方式实现：

- 正常情况下，PC 加 4，各级寄存器在时钟沿装入新的数据和控制信号；
- stall 时保持 PC 和 IF/ID 不变，将 ID/EX 的控制信号置为无效，形成 bubble；
- flush 时将相应级间寄存器写入 NOP 状态，从而撤销错误路径上的指令；
- 复位时将 PC、流水寄存器和写使能信号统一恢复到初始状态。

2.3.2 控制与冒险处理

ForwardingUnit 和 HazardUnit 均采用组合逻辑实现。ForwardingUnit 比较 ID/EX 中的源寄存器与 EX/MEM、MEM/WB 中的目的寄存器，并优先选择距离当前指令更近的 EX/MEM 结果。HazardUnit 则结合 `uses_rs` 和 `uses_rt` 判断当前指令是否真正读取相应字段，避免将立即数中的寄存器编号误判为数据相关。具体策略如表 1 所示。

表 1: 流水线冒险处理策略

情形	判定阶段	工程处理方式
一般数据冒险	EX	从 EX/MEM 或 MEM/WB 转发，且前者优先级更高。
Load-use 数据冒险	ID	保持 PC 和 IF/ID，向 ID/EX 插入 bubble；stall 一个周期后从 MEM/WB 转发加载结果。
条件分支控制冒险	EX	分支成立时更新 PC，并对 IF/ID 和 ID/EX 执行 flush。
`j`/`jal` 控制冒险	ID	在 ID 级形成跳转目标并对 IF/ID 执行 flush。
`jr`/`jalr` 控制冒险	ID	若目标寄存器仍在 ID/EX 或 EX/MEM 中等待写回，则先 stall；数据就绪后完成跳转并对 IF/ID 执行 flush。

store 的写数据和分支比较操作数则共用普通的 forwarding 通路，无需单独增加旁路。控制优先级上，EX 级中较早指令的成立分支优先于 ID 级的较晚跳转；ID 级跳转只有在不存在 stall 时才会生效，从而保证处理器始终按照程序顺序提交有效结果。

2.3.3 存储系统与地址映射

CPU 采用哈佛架构，Instruction ROM 与 Data RAM 使用独立的地址和数据接口。ROM 实例化为 1024×32 bit，RAM 实例化为 4096×32 bit，即 16 KiB。两者均通过 `$readmemh` 加载初始化文件，处理器使用字节地址，以地址的 `[31:2]` 位作 word 索引。

Instruction ROM 采用组合读方式；Data RAM 采用组合读、时钟上升沿写的方式，与流水线 MEM 级的时序相配合。Data RAM 用于保存串口输入、邻接表、DFS 数组、结果、栈和七段码查找表。AmplifierSoC 根据 CPU 给出的数据地址产生 RAM、数码管和 UART 的片选信号，并将被选中设备的读数据复用后送回 CPU。

`0x00000000--0x3FFFFFFF` 为数据 RAM 地址空间，`0x40000000--0x7FFFFFFF` 为外设地址空间。当前工程的实际映射如表 2 所示。

表 2: 数据存储器与外设地址映射

地址（字节地址）	功能	说明
`0x00000000--0x00003FFF`	Data RAM	存放输入数据、运行时数组、结果、栈和七段码表。
`0x40000010`	七段数码管	写入低 12 位，其中低 8 位为段选，高 4 位为位选。
`0x40000018`	UART_TXD	写入低 8 位，发送模块空闲时触发一个字节发送。
`0x4000001C`	UART_RXD	读取最近接收到的字节，返回数据的低 8 位有效。
`0x40000020`	UART_CON	bit 4 为发送忙，bit 3 为接收完成，bit 2 为发送完成；读取后清除完成状态。

2.3.4 七段数码管与 UART 工作原理

七段数码管采用软件译码和软件扫描。Data RAM 的 `0x1A00` 起始位置存放数字 `0--F` 对应的 16 项段码。汇编程序依次提取结果的四个十六进制半字节，以半字节数值作为索引查表，再将所得段码与当前位选信号组合后写入 `0x40000010`。SevenSegPeripheral 只负责锁存软件写入的低 12 位数据，不包含硬件译码器和独立的扫描计数器。

UART 外设由 UartTx、UartRx 和 UartPeripheral 三部分组成。发送器与接收器均采用 IDLE、START、DATA、STOP 状态机；接收端使用两级触发器同步异步输入，并在起始位中部进行确认。系统时钟为 100 MHz，串口波特率为 9600 baud，因此设置 `CLKS_PER_BIT=10417`，通信格式为 8N1，Hex 发送，32 位大端序。

UartPeripheral 将发送数据、接收数据和状态信息映射为三个寄存器，并维护发送完成、接收完成和发送忙标志。汇编程序通过轮询 UART_CON 完成收发同步：输入时每接收四个字节，按大端顺序组合为一个 32 位整数并写入 Data RAM；计算结束后，再将结果拆分为四个大端字节依次写入 UART_TXD。

3 算法指令

3.1 题目要求与总体设计

理论课大作业要求使用 MIPS32 汇编实现给定的节点放大器算法，并保留原算法中的输入处理、循环、分支和递归过程调用。

原始 MARS 程序通过文件相关的 `syscall` 读取 `test.dat`，在模拟器的数据段中保存数组，并通过 `syscall` 输出答案。FPGA 上不存在 MARS 和操作系统，因此新建改编版 `amplifier_fpga.asm`，保留原程序的邻接表构建、递归 DFS 和动态规划逻辑，同时进行如下改造：

- 显式将栈顶设置为 `0x3FFC`，自行维护递归调用的栈帧；
- 使用固定 RAM 地址保存输入流、邻接表、动态规划数组和结果；
- 使用真实 MIPS 指令代替 `li`、`la`、`move` 等伪指令；
- 通过存储器映射 UART 接收测试数据并输出结果，不再使用 `syscall`；
- 通过 RAM 查表完成十六进制七段码译码，并由汇编程序动态扫描数码管。

程序启动后，首先从 UART 接收完整输入并依次写入 RAM；随后初始化数组、构建链式邻接表，从根节点 1 开始递归执行 DFS。算法结束后将结果保存到 RAM，通过 UART 输出 32 位结果，最后进入数码管软件扫描循环。

3.2 输入格式与 RAM 布局

输入数据仍采用理论课大作业规定的整数序列格式：

- 第一个整数为节点数 n；
- 对于节点 1 到 n，依次给出该节点的邻接点数量 k，以及 k 组 `(v,len)`；
- 最后一个整数为信号强度上限 h。

UART 版本将每个整数表示为一个 32 位大端数据。例如整数 4 对应字节序列

00 00 00 04.

接收到的整数首先顺序写入 `0x0000` 起始的输入区，之后再由主程序解析。其余数组均使用固定的字节地址，具体分配如表 3 所示。

数组元素均为 32 位 word，因此访问第 i 个元素时，先通过左移两位计算偏移量 $4i$。边编号从 1 开始，编号 0 作为链表结束标志。输入区共有 128 个 word；对于完整记录无向树双向邻接关系的输入格式，当前布局最多可容纳 $n = 26$ 的测试数据。

表 3: 节点放大器程序 RAM 布局

地址范围或起始地址	数据对象	作用
0x0000--0x01FC	输入整数流	保存 UART 接收到的 n、邻接信息和 h。
0x0200--0x05FC	head[]	保存各节点第一条邻接边编号，0 表示无边。
0x0600--0x09FC	to_arr[]	保存每条边指向的节点编号。
0x0A00--0x0DFC	len_arr[]	保存每条边的长度。
0x0E00--0x11FC	next_arr[]	保存同一节点的下一条边编号。
0x1200--0x15FC	f_arr[]	保存各子树所需的最少放大器数量。
0x1600--0x19FC	g_arr[]	保存各子树向上传递的最大信号距离。
0x1A00--0x1A3C	display_lut[16]	保存十六进制数字 0--F 的七段码。
0x1B00	h_value	保存信号强度上限 h。
0x1B04	result	保存最终计算结果。
0x3FFC	栈顶	递归调用时，栈由高地址向低地址增长。

3.3 链式邻接表构建

程序顺序扫描输入流，为每条输入边分配一个递增的边编号 e，并使用 head、to_arr、len_arr 和 next_arr 构造链式邻接表。插入节点 u 到 v、长度为 len 的边时，执行

$$\begin{aligned} \texttt{to_arr}[e] &= v, \\ \texttt{len_arr}[e] &= \texttt{len}, \\ \texttt{next_arr}[e] &= \texttt{head}[u], \\ \texttt{head}[u] &= e. \end{aligned}$$

这种头插法只需要常数时间，不需要在 RAM 中保存指针，适合使用整数数组实现。

关键汇编代码如下。其中 $s2 保存当前边数，$t0 保存当前节点编号，$t3 和 $t4 分别保存邻接点和边长。

```
    addiu $s2, $s2, 1          # edge_cnt++
    sll   $t5, $s2, 2          # $t5=edge_cnt*4

    addiu $t6, $zero, 0x0600   # $t6=to_arr首地址
    addu  $t7, $t6, $t5        # $t7=&to_arr[edge_cnt]
    sw    $t3, 0($t7)          # to_arr[edge_cnt]=v

    addiu $t6, $zero, 0x0a00   # $t6=len_arr首地址
    addu  $t7, $t6, $t5        # $t7=&len_arr[edge_cnt]
    sw    $t4, 0($t7)          # len_arr[edge_cnt]=len

    sll   $t6, $t0, 2          # $t6=i*4
```

```
    addiu $t7, $zero, 0x0200    # $t7=head首地址
    addu  $t8, $t7, $t6         # $t8=&head[i]
    lw    $t9, 0($t8)           # $t9=head[i]

    addiu $t6, $zero, 0x0e00    # $t6=next_arr首地址
    addu  $t7, $t6, $t5         # $t7=&next_arr[edge_cnt]
    sw    $t9, 0($t7)           # next_arr[edge_cnt]=head[i]

    sw    $s2, 0($t8)           # head[i]=edge_cnt
```

Listing 1: 链式邻接表插入一条边的关键代码

所有输入处理完成后，程序将最后一个整数写入 0x1B00 作为 h，并调用 dfs(1,0,0)，即从根节点 1 开始递归计算。

3.4 递归 DFS 与动态规划

DFS 子程序的三个参数分别通过 $a0、$a1 和 $a2 传入，表示当前节点 u、父节点 parent 和当前节点到父节点的边长。程序遍历 head[u] 指向的链表，并跳过指向父节点的边，以避免在无向树上重复递归。

设 $C(u)$ 为节点 u 除父节点以外的子节点集合，则合并各子树时有

$$f[u] = \sum_{v \in C(u)} f[v], \qquad g[u] = \max_{v \in C(u)} \left(g[v] + \mathrm{len}(u, v) \right).$$

其中，$f[u]$ 表示以 u 为根的子树中所需的最少放大器数量；$g[u]$ 表示该子树向父节点方向传递时，需要考虑的最大累计信号距离。

在递归处理子节点之前，程序首先判断当前边长是否小于 h。若某条边满足 $\mathrm{len} \geq h$，则跳转至 impossible，并令最终结果为 0；否则将子节点、当前节点和边长作为参数，递归调用 dfs。

```
    beq   $s5, $s1, dfs_next_edge  # if(v==parent) continue

    addiu $t1, $zero, 0x1b00       # $t1=h_val地址
    lw    $t2, 0($t1)              # $t2=h
    slt   $t3, $s6, $t2            # 判断len<h
    beq   $t3, $zero, impossible   # 若len>=h, impossible

    addu  $a0, $s5, $zero          # $a0=v
    addu  $a1, $s0, $zero          # $a1=u
    addu  $a2, $s6, $zero          # $a2=len
    jal   dfs                      # 递归处理子结点v
```

Listing 2: 边长检查与 DFS 递归调用

全部子节点处理完毕后，程序根据节点类型更新状态：

- 若 u 为叶节点，则令 $f[u] = 0$、$g[u] = 1$；
- 若 u 不是根节点，且 $g[u] + \text{edge_to_parent} > h$，则必须在当前节点设置一个放大器，因此令 $f[u]$ 加 1，并将 $g[u]$ 重置为 1；
- 其他情况下直接保留子树合并得到的 $f[u]$ 和 $g[u]$；
- 根节点不再检查通向父节点的边，最终答案为 $f[1]$。

下面为汇编程序关键代码：

```
dfs_edge_end:
    beq   $s3, $zero, dfs_leaf      # 若sons==0，当前结点为叶子
    addiu $t0, $zero, 1             # $t0=1，即root编号
    beq   $s0, $t0, dfs_return      # 若u是root，不在root处安装放大器
    sll   $t0, $s0, 2               # $t0=u*4
    addiu $t1, $zero, 0x1600        # $t1=g_arr首地址
    addu  $t2, $t1, $t0             # $t2=&g[u]
    lw    $t3, 0($t2)               # $t3=g[u]
    addu  $t3, $t3, $s2             # $t3=g[u]+edge_to_parent
    addiu $t4, $zero, 0x1b00        # $t4=h_val地址
    lw    $t5, 0($t4)               # $t5=h
    slt   $t6, $t5, $t3             # 判断h<g[u]+edge_to_parent
    beq   $t6, $zero, dfs_return    # 若不超过h，不需要放大器
    addiu $t1, $zero, 0x1200        # $t1=f_arr首地址
    addu  $t2, $t1, $t0             # $t2=&f[u]
    lw    $t3, 0($t2)               # $t3=f[u]
    addiu $t3, $t3, 1               # f[u]++
    sw    $t3, 0($t2)               # 保存f[u]
    addiu $t1, $zero, 0x1600        # $t1=g_arr首地址
    addu  $t2, $t1, $t0             # $t2=&g[u]
    addiu $t3, $zero, 1             # $t3=1
    sw    $t3, 0($t2)               # g[u]=1
    j     dfs_return                # 返回调用者

dfs_leaf:
    sll   $t0, $s0, 2               # $t0=u*4
    addiu $t1, $zero, 0x1200        # $t1=f_arr首地址
    addu  $t2, $t1, $t0             # $t2=&f[u]
    sw    $zero, 0($t2              # f[u]=0
    addiu $t1, $zero, 0x1600        # $t1=g_arr首地址
    addu  $t2, $t1, $t0             # $t2=&g[u]
    addiu $t3, $zero, 1             # $t3=1
    sw    $t3, 0($t2)               # g[u]=1
```

Listing 3: DFS 节点状态判断与更新

由于 DFS 使用递归过程调用，每次进入 dfs 时在栈中分配 36 Byte 空间，保存 $s0--$s7 和返回地址 $ra；返回前再按相反顺序恢复现场，避免干扰。

3.5 UART 输入输出的裸机实现

UART 输入程序先轮询 0x40000020 的 bit 3。接收完成标志有效时. 从 0x4000001C 读低 8 位数据。四次调用 uart_recv_byte 后按大端序将四个字节合成一个 32 位整数。

```
    jal   uart_recv_byte          # 接收第1个字节
    sll   $s0, $v0, 24            # 放入结果的31--24位
    jal   uart_recv_byte          # 接收第2个字节
    sll   $t0, $v0, 16            # 放入结果的23--16位
    or    $s0, $s0, $t0           # 合并前两个字节
    jal   uart_recv_byte          # 接收第3个字节
    sll   $t0, $v0, 8             # 放入结果的15--8位
    or    $s0, $s0, $t0           # 合并第3个字节
    jal   uart_recv_byte          # 接收第4个字节
    or    $v0, $s0, $v0           # 合并低8位并返回32位整数
```

Listing 4: 四个 UART 字节组合为 32 位整数

单字节接收代码使用 lui 和 ori 构造外设地址，并通过 lw 读取状态和接收数据。

```
uart_recv_byte:
    lui   $t6, 0x4000             # 构造UART_CON高16位
    ori   $t6, $t6, 0x0020        # $t6=0x40000020, UART_CON
    lui   $t5, 0x4000             # 构造UART_RXD高16位
    ori   $t5, $t5, 0x001c        # $t5=0x4000001C, UART_RXD

uart_recv_wait_ready:
    lw    $t4, 0($t6)             # 读取UART状态寄存器
    andi  $t4, $t4, 0x0008        # 取bit 3接收完成标志
    beq   $t4, $zero, uart_recv_wait_ready # 未接收完成则继续等待
    lw    $v0, 0($t5)             # 读取接收到的数据
    andi  $v0, $v0, 0x00ff        # 仅保留低8位
    jr    $ra                     # 返回调用者
```

Listing 5: UART 单字节轮询接收

发送过程与接收过程相反。程序依次提取结果的第 31–24、23–16、15–8 和 7–0 位，并分别调用 uart_send_byte。单字节发送前轮询 UART_CON 的 bit 4，只有发送器空闲时才向 UART_TXD 写入数据。

```
uart_send_byte:
    lui   $t6, 0x4000             # 构造UART_CON高16位
    ori   $t6, $t6, 0x0020        # $t6=0x40000020, UART_CON
    lui   $t5, 0x4000             # 构造UART_TXD高16位
    ori   $t5, $t5, 0x0018        # $t5=0x40000018, UART_TXD

```

```
uart_send_byte_wait:
    lw    $t4, 0($t6)                # 读取UART状态寄存器
    andi  $t4, $t4, 0x0010           # 取bit 4发送忙标志
    bne   $t4, $zero, uart_send_byte_wait # 发送器忙则继续等待
    sw    $t3, 0($t5)                # 写入低8位并触发发送
    jr    $ra                        # 返回调用者
```

Listing 6: UART 单字节轮询发送

UART 接收函数根据输入中的 n 和各节点的邻接点数量判断还需要接收多少个整数，因此不依赖固定的串口字节总数。完整输入接收结束后，算法才开始运行；结果通过 UART 发送完成后，程序再进入数码管显示循环。

3.6 结果保存与软件七段译码

正常情况下，DFS 返回后从 `f_arr[1]` 取得答案；若发现不可满足的边，则将答案置为 0。程序将结果写入 `0x1B04`，随后调用 `uart_send_word32` 输出完整的 32 位结果。

数码管显示使用 `0x1A00` 起始的 16 项七段码查找表。对于结果的每个十六进制半字节 d，程序计算 $A_{\mathrm{LUT}} = \texttt{0x1A00} + 4d$，从 RAM 中读取已经译好的段码，再与 `0x0100`、`0x0200`、`0x0400` 或 `0x0800` 位选信号组合，写入数码管外设地址 `0x40000010`。

以下代码给出了结果保存以及最低一位数码管的查表显示过程，其余三位同理。

```
program_done:
    addiu $t0, $zero, 0x1b04       # $t0=result地址
    sw    $a0, 0($t0)              # 保存最终结果
    jal   uart_send_word32         # 通过UART发送32位结果

display_loop:
    addiu $s6, $zero, 0x1b04       # $s6=result地址
    lw    $s7, 0($s6)              # $s7=最终结果
    addiu $s5, $zero, 0x1a00       # $s5=display_lut首地址
    lui   $s4, 0x4000              # 构造数码管外设地址高16位
    ori   $s4, $s4, 0x0010         # $s4=0x40000010
    andi  $t0, $s7, 0x000f         # 提取结果最低4位
    sll   $t1, $t0, 2              # 七段码表索引乘4
    addu  $t1, $s5, $t1            # $t1=&display_lut[d]
    lw    $t2, 0($t1)              # 读取数字d对应的七段码
    ori   $t2, $t2, 0x0100         # 加入最低位数码管位选
    sw    $t2, 0($s4)              # 写入段码和位选
    jal   delay                    # 软件延时后切换显示位
```

Listing 7: 结果保存与软件七段码查表

每显示一位后，程序调用 `delay` 产生软件延时，再切换到下一位。四位显示完成后重新从最低位开始扫描。整个译码、位选和刷新过程均由 MIPS 汇编程序完成，Verilog 外设只锁存软件写入的 12 位段选和位选数据，符合软件实现七段译码的实验要求。

4 工程文件清单与关键代码

4.1 工程目录结构

本节仅介绍包含 UART 输入输出功能的提高版工程。基础版与提高版使用相同的流水线 CPU 核心，因此本节不再重复介绍基础版。工程目录结构如下：

```
pipeline_cpu_project_uart_bonus/
|-- rtl/           # .v工程文件，实现CPU、存储器、外设和顶层RTL
|-- sim/           # .v仿真文件，用于UART单元测试和完整SoC仿真
|-- constraints/   # .xdc约束文件，时钟、复位、UART和数码管管脚约束
|-- asm/           # 节点放大器裸机汇编程序及机器码
|-- data/          # Data RAM初值和UART样例输入
|-- BoardTop.bit   # .bit可烧录二进制文件
```

4.2 rtl 文件夹

注：后仿时临时的顶层文件 BoardTop_sim.v 请见 5.4。

rtl 文件夹包含 CPU 核心、指令与数据存储器、数码管和 UART 外设，以及 SoC 和板级顶层模块。各模块采用分层方式连接，最终由 BoardTop 作为 Vivado 综合顶层。

4.2.1 BoardTop.v：顶层模块

BoardTop 是上板综合使用的顶层模块，其端口名称与 XDC 约束保持一致。该模块不包含处理器逻辑，只负责例化 AmplifierSoC，连接 100 MHz 时钟、复位、UART 和四位七段数码管，并隐藏仅用于仿真的调试端口。

```
AmplifierSoC soc(
    .clk                  (sysclk),
    .reset                (KEY1),
    .uart_rx              (UART_RX),
    .uart_tx              (UART_TX),
    .seg                  (leds),
    .an                   (SEL),
    .debug_pc             (debug_pc_unused),
    .debug_sevenseg_value (debug_sevenseg_unused)
);
```

4.2.2 AmplifierSoC.v：集成模块

AmplifierSoC 是完整片上系统的集成模块，例化 PipelineCPU、Instruction ROM、Data RAM、七段数码管外设和 UART 外设。该模块对 CPU 的数据地址进行组合译码，并将被选中设备的读数据复用回 CPU。

RAM、数码管和 UART 的写使能均由 CPU 总写使能与对应片选信号共同产生，避免一次访存操作同时改变多个设备。

```
assign ram_selected      = (data_addr[31:30] == 2'b00);
assign sevenseg_selected = (data_addr == SEVENSEG_ADDR);
assign uart_selected     = (data_addr == UART_TXD ||
                            data_addr == UART_RXD ||
                            data_addr == UART_CON);
assign data_rdata = ram_selected  ? ram_rdata :
                    uart_selected ? uart_rdata : 32'b0;
```

4.2.3 PipelineCPU.v：综合实现 CPU 数据通路

PipelineCPU 实现 IF、ID、EX、MEM、WB 五级流水线以及四组级间寄存器。模块内部连接控制器、寄存器堆、ALU、ForwardingUnit 和 HazardUnit，并向 SoC 提供相互独立的指令接口和数据接口。

下一 PC 按照“EX 级成立分支、ID 级有效跳转、顺序执行”的优先级选择；stall 时禁止 PC 更新，而成立分支始终具有最高控制优先级。

```
assign pc_next = ex_branch_taken ? ex_branch_target :
                 id_jump_take    ? id_jump_target :
                                   pc4;
assign pc_write = ex_branch_taken || !stall;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc <= 32'b0; end
    else if (pc_write) begin
        pc <= pc_next; end
end
```

IF/ID 和 ID/EX 寄存器根据 branch、jump 和 stall 条件选择正常更新、保持或写入 bubble；EX/MEM 和 MEM/WB 则在每个时钟沿正常向后传递数据与控制信号。

4.2.4 ControlUnit.v：译码生成控制信号

ControlUnit 根据 opcode、funct 和 rt 字段组合译码，产生寄存器写回、访存、ALU、立即数扩展、分支、跳转和源寄存器使用情况等控制信号。

作为示例，以下代码展示了 lw 和 sw 的译码过程。

```
6'h23: begin // lw
    reg_write    = 1'b1;
    mem_read     = 1'b1;
    result_src   = RES_MEM;
    alu_src2_imm = 1'b1;
    alu_ctl      = ALU_ADD;
    uses_rs      = 1'b1;
end
```

```
6'h2b: begin // sw
    mem_write    = 1'b1;
    alu_src2_imm = 1'b1;
    alu_ctl      = ALU_ADD;
    uses_rs      = 1'b1;
    uses_rt      = 1'b1;
end
```

除节点放大器程序实际使用的指令外，控制器还实现了指导书要求的条件分支和跳转指令。虽然文件中保留了 `mul` 的译码入口，但实际 ALU 实例关闭了组合乘法通路。

4.2.5 ALU.v：计算模块

`ALU` 是 EX 级的主要组合运算模块。其根据五位 `alu_ctl` 完成算术、逻辑、移位和有符号/无符号比较，并输出运算结果及零标志。

```
always @(*) begin
    case (alu_ctl)
        ALU_AND:  out = in1 & in2;
        ALU_OR:   out = in1 | in2;
        ALU_ADD:  out = in1 + in2;
        ALU_SUB:  out = in1 - in2;
        ALU_SLT:  out = ($signed(in1) < $signed(in2)) ?
                        32'd1 : 32'd0;
        ALU_NOR:  out = ~(in1 | in2);
        ALU_XOR:  out = in1 ^ in2;
        ALU_SLL:  out = in2 << in1[4:0];
        ALU_SRL:  out = in2 >> in1[4:0];
        ALU_SRA:  out = $signed(in2) >>> in1[4:0];
        ALU_SLTU: out = (in1 < in2) ? 32'd1 : 32'd0;
        default:  out = 32'b0;
    endcase
end
```

4.2.6 RegisterFile.v：实现通用寄存器

`RegisterFile` 实现 32 个 MIPS 通用寄存器，其中 0 号寄存器始终读出零且禁止写入。寄存器堆采用双端口组合读和时钟上升沿写，并设置 WB 到 ID 的同周期旁路。

```
assign read_data1 = (read_reg1 == 5'd0) ? 32'b0 :
    ((reg_write && write_reg == read_reg1 &&
      write_reg != 5'd0) ? write_data : regs[read_reg1]);

assign read_data2 = (read_reg2 == 5'd0) ? 32'b0 :
    ((reg_write && write_reg == read_reg2 &&
      write_reg != 5'd0) ? write_data : regs[read_reg2]);

```

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 1; i < 32; i = i + 1)
            regs[i] <= 32'b0;
    end else if (reg_write && write_reg != 5'd0) begin
        regs[write_reg] <= write_data;
    end
end
```

4.2.7 ForwardingUnit.v：实现数据转发

ForwardingUnit 比较 ID/EX 中的两个源寄存器与 EX/MEM、MEM/WB 的目的寄存器，分别产生 forward_a 和 forward_b。当两个后级均可提供数据时，优先选择距离当前指令更近的 EX/MEM 结果。

```
always @(*) begin
    forward_a = 2'b00;
    forward_b = 2'b00;

    if (ex_mem_reg_write && ex_mem_can_forward &&
        ex_mem_write_reg != 5'd0 &&
        ex_mem_write_reg == id_ex_rs)
        forward_a = 2'b10;
    else if (mem_wb_reg_write &&
             mem_wb_write_reg != 5'd0 &&
             mem_wb_write_reg == id_ex_rs)
        forward_a = 2'b01;

    if (ex_mem_reg_write && ex_mem_can_forward &&
        ex_mem_write_reg != 5'd0 &&
        ex_mem_write_reg == id_ex_rt)
        forward_b = 2'b10;
    else if (mem_wb_reg_write &&
             mem_wb_write_reg != 5'd0 &&
             mem_wb_write_reg == id_ex_rt)
        forward_b = 2'b01;
end
```

4.2.8 HazardUnit.v：检测并实行阻塞

HazardUnit 检测不能仅通过转发解决的冒险。其主要处理紧邻的 load-use 数据相关，并在 jr 或 jalr 的目标寄存器尚未写回时暂停流水线前端。

```
assign load_use_rs =
    id_ex_mem_read && if_id_uses_rs &&
    id_ex_rt != 5'd0 && id_ex_rt == if_id_rs;
```

```
assign load_use_rt =
    id_ex_mem_read && if_id_uses_rt &&
    id_ex_rt != 5'd0 && id_ex_rt == if_id_rt;

assign jump_reg_wait_ex =
    if_id_jump_reg && id_ex_reg_write &&
    id_ex_write_reg != 5'd0 &&
    id_ex_write_reg == if_id_rs;

assign jump_reg_wait_mem =
    if_id_jump_reg && ex_mem_reg_write &&
    ex_mem_write_reg != 5'd0 &&
    ex_mem_write_reg == if_id_rs;

assign stall = load_use_rs || load_use_rt ||
               jump_reg_wait_ex || jump_reg_wait_mem;
```

4.2.9 InstructionROM.v：指令存储器

InstructionROM 使用 Verilog 数组实现 1024×32 bit 指令存储器。仿真和综合开始时通过 $readmemh 载入机器码文件，组合逻辑读取，超出 ROM 范围时返回 NOP。

```
initial begin
    for (i = 0; i < ROM_WORDS; i = i + 1)
        rom[i] = 32'h00000000;
    $readmemh(INIT_FILE, rom);
end

assign instr = (addr[31:2] < ROM_WORDS) ?
               rom[addr[31:2]] : 32'h00000000;
```

4.2.10 DataRAM.v：数据存储器

DataRAM 使用 Verilog 数组实现 4096×32 bit 数据存储器。模块先将 RAM 清零，再加载 amplifier_data.hex。读取采用组合逻辑，写入在时钟上升沿完成。

```
initial begin
    for (i = 0; i < RAM_WORDS; i = i + 1)
        ram[i] = 32'h00000000;
    $readmemh(INIT_FILE, ram);
end

assign read_data =
    (mem_read && addr[31:2] < RAM_WORDS) ?
    ram[addr[31:2]] : 32'h00000000;

```

```
always @(posedge clk) begin
    if (mem_write && addr[31:2] < RAM_WORDS)
        ram[addr[31:2]] <= write_data;
end
```

4.2.11 SevenSegPeripheral.v：数码管寄存器

SevenSegPeripheral 是映射到 0x40000010 的数码管寄存器。模块只锁存 CPU 写入数据的低 12 位，其中低 8 位连接段选，高 4 位连接位选，不包含硬件七段译码或自动扫描逻辑。

```
reg [11:0] value;

assign raw_value = value;
assign seg = value[7:0];
assign an  = value[11:8];

always @(posedge clk or posedge reset) begin
    if (reset)
        value <= 12'b0;
    else if (mem_write)
        value <= write_data[11:0];
end
```

4.2.12 UartTx.v：UART 发送

UartTx 使用 IDLE、START、DATA、STOP 四状态有限状态机发送 UART 字节。CLKS_PER_BIT 决定每一位保持的时钟周期数，数据位按照低位优先顺序发送。

```
S_DATA: begin
    tx <= tx_data[bit_index];
    if (clk_count == CLKS_PER_BIT - 1) begin
        clk_count <= 32'b0;
        if (bit_index == 3'd7) begin
            bit_index <= 3'b0;
            state     <= S_STOP;
        end else begin
            bit_index <= bit_index + 3'd1;
        end
    end else begin
        clk_count <= clk_count + 32'd1;
    end
end
```

发送开始后 busy 保持为 1，停止位发送完成时产生单周期 done 脉冲，并重新回到空闲状态。

4.2.13 UartRx.v：UART 接收

UartRx 同样采用四状态有限状态机。异步串口输入首先经过两级触发器同步；检测到低电平后，在半个 bit 周期处确认起始位，再每隔一个 bit 周期采样八个数据位。

```
S_DATA: begin
    if (clk_count == CLKS_PER_BIT - 1) begin
        clk_count       <= 32'b0;
        data[bit_index] <= rx_sync;
        if (bit_index == 3'd7) begin
            bit_index <= 3'b0;
            state     <= S_STOP;
        end else begin
            bit_index <= bit_index + 3'd1;
        end
    end else begin
        clk_count <= clk_count + 32'd1;
    end
end
```

停止位计时结束后，模块产生单周期 done 信号，通知上层外设一个完整字节已经接收。

4.2.14 UartPeripheral.v：UART 相关例化

UartPeripheral 例化 UartTx 和 UartRx，并将其包装为三个存储器映射寄存器：UART_TXD、UART_RXD 和 UART_CON。组合读逻辑如下：

```
always @(*) begin
    case (addr)
        UART_RXD:
            read_data = {24'b0, rx_data_reg};
        UART_CON:
            read_data = {27'b0, tx_busy,
                         rx_ready, tx_done_flag, 2'b00};
        default:
            read_data = 32'b0;
    endcase
end
```

写 UART_TXD 时，模块在发送器空闲的条件下产生一个周期的 tx_start；读取 UART_CON 后清除发送完成和接收完成标志。新接收事件具有更高更新优先级，避免状态清除与接收完成发生在同一周期时丢失新数据。

注：UART 参数和输入协议请见2.3.4。

4.3 sim 文件夹

注： 后仿代码 tb_board_post_impl.v 请见 5.4，用于 CPI 计数的仿真文件副本 tb_amplifier_soc_cpi.v 请见 6.4。

sim 文件夹包含 UART 外设单元测试、完整 SoC 仿真以及相应的仿真文件列表。仿真时缩小 CLKS_PER_BIT 以减少运行时间，但 UART 状态机和串行协议保持不变。

4.3.1 tb_amplifier_soc.v：完整 SoC 仿真

例化完整 AmplifierSoC，通过 UART RX 输入节点放大器试样例，并同时检查 RAM 结果、UART 输出和数码管扫描值。仿真中将 CLKS_PER_BIT 改为 16。

```
for (cycle = 0; cycle < 200000;
     cycle = cycle + 1) begin
    @(posedge clk);
    if (dut.data_ram.ram[RESULT_WORD] == 32'd1 &&
        uart_count >= 4 &&
        uart_word == 32'h00000001 &&
        debug_sevenseg_value == 12'h83f) begin
        $display("PASS: result=%0d uart=%h sevenseg=%h cycles=%0d pc=%h",
            dut.data_ram.ram[RESULT_WORD],
            uart_word, debug_sevenseg_value, cycle, debug_pc);
        $finish;
    end
end
```

测试平台还实现了 UART 字节发送、32 位大端整数发送和 UART TX 解码任务，从而不依赖额外串口模型。

4.3.2 tb_uart_peripheral.v：UART 外设仿真

对 UART 外设做独立 smoke test。先通过总线向 UART_TXD 写入 0x41，再从串行输出线上解码；再向 RX 输入 0x5A，检查接收数据及 UART_CON 的状态清除行为。

```
bus_write(UART_TXD, 32'h00000041);
wait_uart_tx_byte(decoded_tx);
if (decoded_tx != 8'h41) begin
    $display("FAIL: UART TX expected 0x41, got 0x%02h", decoded_tx);
    $finish;
end
drive_uart_rx_byte(8'h5a);
repeat (4) @(posedge clk);
bus_read(UART_RXD, read_value);
if (read_value[7:0] != 8'h5a) begin
    $display("FAIL: UART RX expected 0x5a, got 0x%02h", read_value[7:0]);
    $finish;
end
```

4.4 constraints 文件夹

4.4.1 welog1_pipeline_cpu.xdc：管脚约束

该文件是 BoardTop 对应的板级约束，目标器件为课程实验板上的 xc7a35tfgg484-1。约束文件将 100 MHz 时钟、KEY1 高有效复位、USB-UART、八个段选和四个位选信号绑定到实际管脚，并创建 10 ns 时钟约束。

```
set_property -dict {
    PACKAGE_PIN R4 IOSTANDARD LVCMOS33
} [get_ports {sysclk}]
create_clock -name sysclk -period 10.000 \
    [get_ports {sysclk}]

set_property -dict {
    PACKAGE_PIN B22 IOSTANDARD LVCMOS33
} [get_ports {KEY1}]

set_property -dict {
    PACKAGE_PIN B20 IOSTANDARD LVCMOS33
} [get_ports {UART_RX}]
set_property -dict {
    PACKAGE_PIN A20 IOSTANDARD LVCMOS33
} [get_ports {UART_TX}]

set_property -dict {
    PACKAGE_PIN N2 IOSTANDARD LVCMOS33
} [get_ports {leds[0]}]
set_property -dict {
    PACKAGE_PIN Y3 IOSTANDARD LVCMOS33
} [get_ports {SEL[0]}]
```

其余段选和位选按照实验板的物理顺序依次约束。逻辑 SEL[0] 对应最右侧最低位，SEL[3] 对应最左侧最高位；输出端统一设置为 SLEW SLOW 和 DRIVE 8。

4.5 asm 文件夹

4.5.1 amplifier_fpga.asm：裸机汇编

该文件是处理器实际执行的节点放大器裸机汇编程序。它保留理论课程序的邻接表和递归 DFS 算法，删除 MARS `syscall`，改为从 UART 接收输入、访问固定 RAM 地址、发送计算结果，并通过软件查表扫描数码管。第三节已经对其算法和关键代码进行了详细说明。

程序入口首先设置栈顶并接收完整输入，随后从 RAM 起始地址读取节点数。

```
main:
    addiu $sp, $zero, 0x3ffc
    jal   uart_receive_input

    addiu $s3, $zero, 0x0000
    lw    $s0, 0($s3)
    addiu $s3, $s3, 4

    addiu $t0, $zero, 0
```

该汇编程序共生成 346 条机器指令，最终写入 Instruction ROM。程序执行结束后不会退出，而是持续运行数码管软件扫描循环。

4.5.2 amplifier_program.hex：指令机器码

该文件由工程汇编器根据 `amplifier_fpga.asm` 生成，每行保存一个 32 位机器码 word，并由 Instruction ROM 中的 `$readmemh` 加载。文件开头如下：

```
241d3ffc
0c000068
24130000
8e700000
26730004
24080000
0208c82a
1720000c
00084880
240a0200
```

其中第一条机器码 `0x241D3FFC` 对应 MIPS 指令 `addiu $sp,$zero,0x3FFC`，用于初始化裸机递归栈。

4.6 data 文件夹

4.6.1 amplifier_data.hex：初始化 RAM

该文件用于初始化 4096×32 bit Data RAM。UART 提高版不在 RAM 中预置测试输入，因此除七段码查找表外，其余位置初始为零。地址 `0x1A00` 起的 16 个 word 依次保存数字 `0--F` 的段码：

```
0000003f
00000006
0000005b
0000004f
00000066
0000006d
0000007d
00000007
0000007f
0000006f
00000077
0000007c
00000039
0000005e
00000079
00000071
```

汇编程序以十六进制半字节作为索引读取该表，再与位选信号组合后写入数码管外设，因此 Verilog 中不需要数字译码器。

4.6.2 uart_sample_input_bytes.hex：测试样例

该文件保存节点放大器样例的 UART 输入字节。每行表示一个 32 位大端整数，串口调试助手应使用 Hex 模式按顺序发送，不能附加回车或换行。

```
00 00 00 04
00 00 00 02
00 00 00 02
00 00 00 03
00 00 00 03
00 00 00 01
00 00 00 02
00 00 00 01
```

完整文件包含 18 个 32 位整数，共 72 Byte。其数据格式与理论课节点放大器样例一致，发送完成后处理器计算结果应为 1，并通过 UART 返回 `00 00 00 01`。

5 仿真结果与分析

5.1 仿真环境与验证方案

在 Vivado 中进行仿真验证，系统时钟周期设为 10 ns，即工作频率为 100 MHz。验证过程按照“外设单元—完整 SoC—实现后网表”三个层次展开，如表 4 所示。

表 4: 仿真验证方案

验证层次	仿真对象	激励方式	主要判据
UART 前仿	`UartPeripheral`	总线读写及串行输入	字节发送、接收及状态位清除正确
完整 SoC 前仿	`AmplifierSoC`	通过 UART 输入节点放大器测试数据	RAM 结果、UART 输出及数码管显示均正确
实现后时序仿真	`BoardTop_sim` 实现后网表	通过板级 UART 端口输入相同数据	UART 返回结果及数码管端口输出正确

实际 FPGA 运行时，测试数据由 PC 串口助手通过 `UART_RX` 输入；仿真时没有连接实验板，也没有真实串口数据源，因此由 testbench 代替 PC 串口助手产生输入。具体而言，`drive_uart_rx_byte` 任务按照“起始位、8 位数据、停止位”的格式直接驱动 `uart_rx`，`drive_uart_rx_word32` 再按照高字节优先的顺序连续发送 4 个字节。对于输出，testbench 在检测到 `uart_tx` 起始沿后，于每个数据位中部采样，并重新拼接为 32 位结果。因此，仿真输入仍经过完整的 UART 接收模块和存储映射接口，而不是直接写入 CPU 内部寄存器或 RAM。

为缩短仿真时间，UART 单元前仿将 `CLKS_PER_BIT` 设为 8，完整 SoC 前仿和后时序仿真将其设为 16；实际上板版本仍采用 10417，以在 100 MHz 时钟下实现约 9600 波特率。上述修改只用于减少串行数据传输所需的仿真周期。

5.2 UART 外设仿真：前仿 1

UART 外设前仿使用 `tb_uart_peripheral.v`，直接对 UART 的三个存储映射地址进行读写。测试首先向 `0x40000018` 写入 `0x41`，检查发送端输出；随后由 testbench 向接收端发送 `0x5A`，读取 `0x4000001C` 和状态寄存器 `0x40000020`，并检查接收状态位能否在读操作后自动清零。

图 3 （转下页）给出了发送操作的局部波形。在 `mem_write` 有效时，总线地址为 `0x40000018`，写入数据低 8 位为 `0x41`。写操作触发发送状态机，随后 `tx_busy` 有效，`uart_tx` 开始输出对应的串行帧。

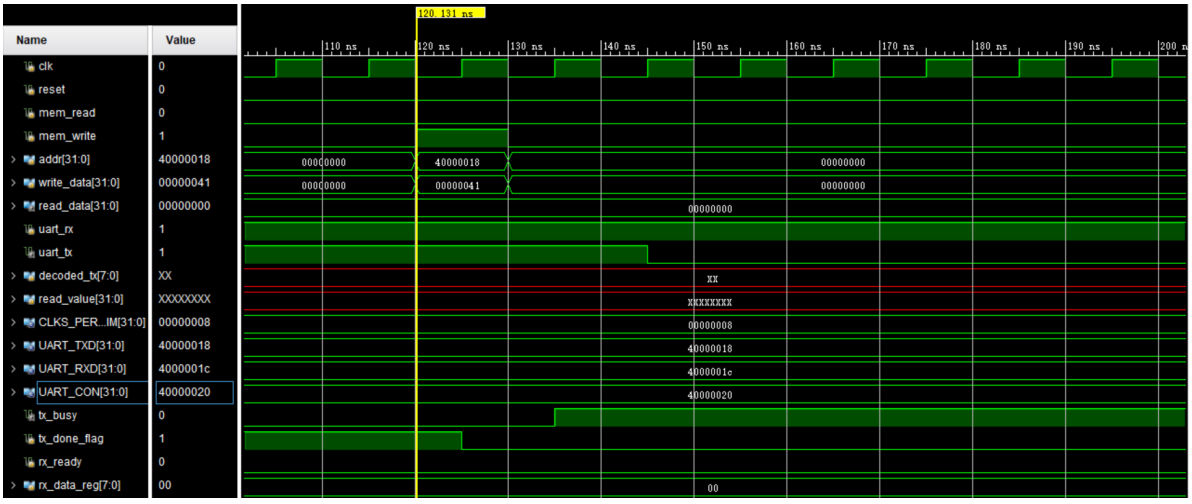


图 3: UART 外设发送操作波形

完整收发过程如图 4 所示。发送监视器最终将 `uart_tx` 解码为 `0x41`；在接收测试中，`uart_rx` 上的串行数据被还原为 `0x5A`，`rx_data_reg` 得到正确结果，`rx_ready` 随后置位。对状态寄存器进行读操作后，接收状态被清除，说明 UART 数据通路及状态管理逻辑均符合设计要求。

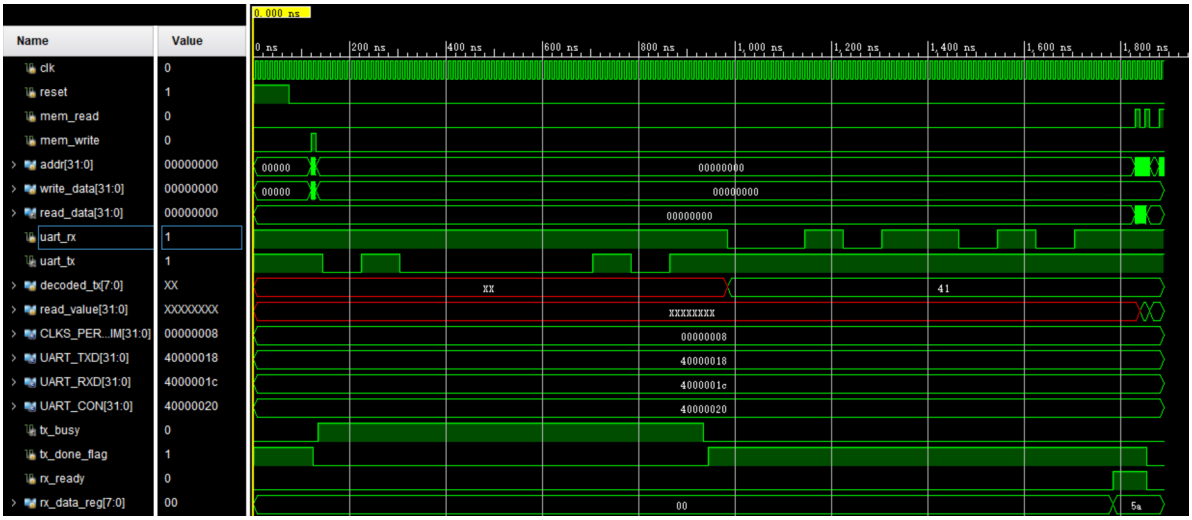


图 4: UART 外设完整收发及状态清除波形

Tcl Console 的自动检查结果如图 5 所示。仿真在 1890 ns 时输出 `PASS: UART peripheral TX/RX`，表明发送数据、接收数据以及状态位清除三项检查均已通过。

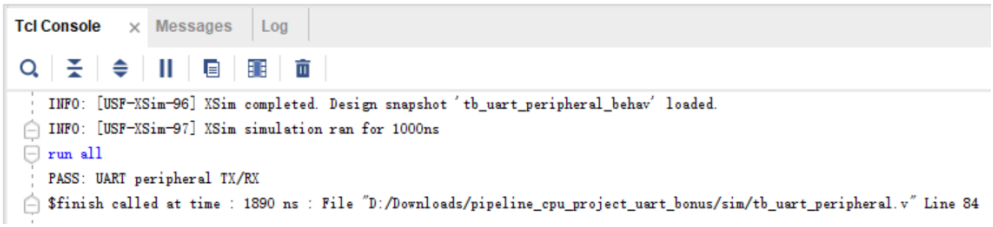


图 5: UART 外设前仿自动检查结果

5.3 完整 SoC RTL 功能仿真：前仿 2

完整 SoC 前仿使用 `tb_amplifier_soc.v`，仿真对象包括流水线 CPU、指令 ROM、数据 RAM、UART 和七段数码管外设。仿真开始时先检查 ROM 与 RAM 初始化是否成功，再通过 UART 输入 18 个 32 位整数。对于该组节点放大器样例，预期计算结果为 1，因此 testbench 同时检查以下三个条件：结果地址 `0x1B04` 中的数据为 1，UART 返回值为 `0x00000001`，数码管扫描状态出现 `0x83F`。

图 6 展示了完整输入输出过程。复位结束后，`uart_rx` 连续输入测试数据；输入完成并执行节点放大器程序后，`uart_tx` 返回四个字节，`uart_count` 增加到 4，拼接结果 `uart_word` 为 `0x00000001`。随后数码管输出由初始值进入软件扫描过程。

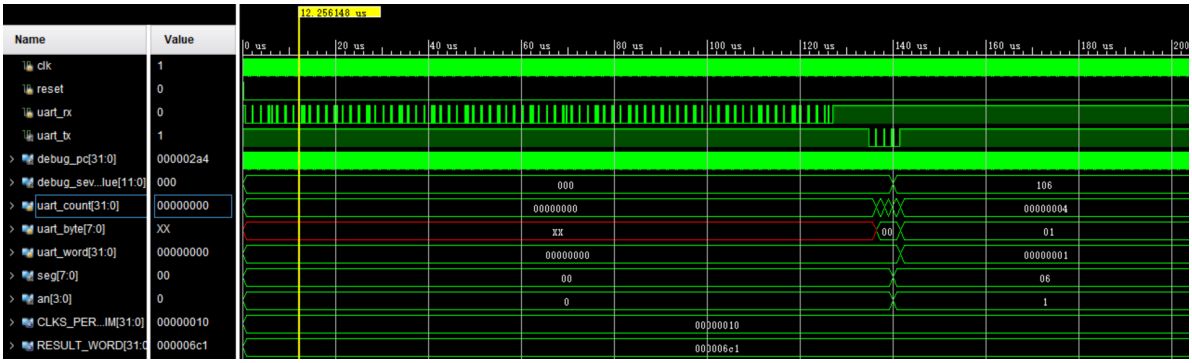


图 6: 完整 SoC RTL 仿真的输入输出波形

图 7 放大了最终结果附近的波形。此时 UART 已接收到四个输出字节，`uart_word` 保持为 `0x00000001`；处理器对数码管地址 `0x40000010` 执行写操作，写入数据由 `0x43F` 更新为 `0x83F`。其中高 4 位为位选，低 8 位为软件查表得到的段码，表明程序正在依次扫描四位数码管并显示结果 `0001`。

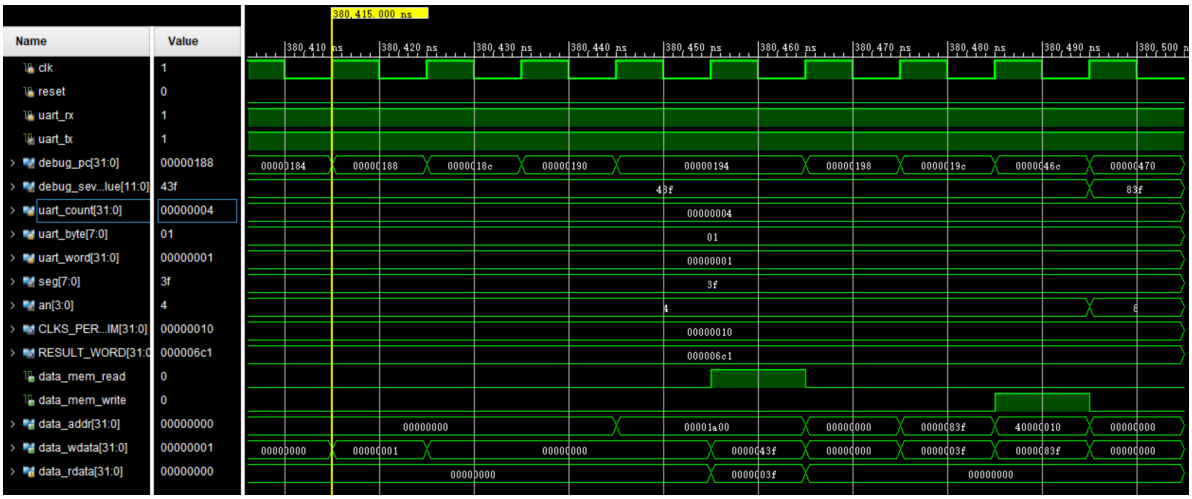


图 7: 完整 SoC RTL 仿真的最终结果波形

自动检查结果如图 8 （转下页）所示。UART 依次输出 `00 00 00 01`，RAM 结果为 1，数码管状态为 `83F`；testbench 在复位后的第 38038 个周期输出 `PASS`，仿真结束时间为 380.505 μs。三种相互独立的输出均与理论结果一致，说明流水线 CPU 能够正确执行节点放大器汇编程序，存储器和外设也能协同工作。



图 8: 完整 SoC RTL 仿真自动检查结果

除图中所示样例外，还使用 ChatGPT 生成了多组不同规模和拓扑结构的合法测试数据，并在相同的仿真框架下重复验证，所得结果均与预期一致。

5.4 实现后时序仿真

后仿使用 Vivado 实现后的门级网表，并反标布局布线产生的 SDF 延迟。由于实现后网表中的模块层次和信号名称可能被展开或重命名，后仿 testbench 不再访问 CPU、ROM 或 RAM 的内部信号，而只通过 `sysclk`、`KEY1`、`UART_RX`、`UART_TX`、`leds` 和 `SEL` 等板级端口进行激励与判定。

实际上板顶层 `BoardTop` 使用 `CLKS_PER_BIT=10417`。若在实现后时序仿真中使用真实 UART 参数，仅发送 72 个输入字节就需要约 82.5 ms 的仿真时间，门级仿真的运行开销过大。为此，新建仅供后仿使用的 `BoardTop_sim`，保持板级端口和其余结构不变，只将 UART 每位时钟数改为 16，如下所示。该顶层重新进行综合与实现；实际上板和生成 bitstream 时仍使用原 `BoardTop`。

```
module BoardTop_sim(
    input         sysclk,
    input         KEY1,
    input         UART_RX,
    output        UART_TX,
    output [7:0] leds,
    output [3:0] SEL
);
    localparam integer CLKS_PER_BIT_SIM = 16;
    wire [31:0] debug_pc_unused;
    wire [11:0] debug_sevenseg_unused;
    AmplifierSoC #(
        .CLKS_PER_BIT(CLKS_PER_BIT_SIM)
    ) soc (
        .clk                    (sysclk),
        .reset                  (KEY1),
        .uart_rx                (UART_RX),
        .uart_tx                (UART_TX),
        .seg                    (leds),
        .an                     (SEL),
```

```
        .debug_pc              (debug_pc_unused),
        .debug_sevenseg_value (debug_sevenseg_unused)
    );
endmodule
```

Listing 8: 后仿专用顶层中的 UART 参数覆盖

后仿 testbench 使用与实现后网表一致的 160 ns UART 位周期，并发送与前仿相同的测试数据。它将连续收到的四个字节重新拼接为 32 位结果，同时等待数码管端口出现 0x83F；只有两项条件同时满足时才输出 PASS。关键判定代码如下。

```
localparam integer CLK_PERIOD_NS     = 10;
localparam integer CLKS_PER_BIT_SIM  = 16;
localparam integer BIT_TIME_NS       = CLK_PERIOD_NS * CLKS_PER_BIT_SIM;
localparam integer TIMEOUT_NS        = 2000000;
localparam [31:0] EXPECTED_UART_WORD = 32'h00000001;
localparam [11:0] EXPECTED_SEVENSEG  = 12'h83f;

BoardTop_sim dut (
    .sysclk  (sysclk),
    .KEY1    (KEY1),
    .UART_RX (UART_RX),
    .UART_TX (UART_TX),
    .leds    (leds),
    .SEL     (SEL)
);

initial begin
    wait (KEY1 == 1'b0);
    wait (uart_count >= 4);
    wait ({SEL, leds} === EXPECTED_SEVENSEG);
    seen_expected_sevenseg = 1'b1;
end

initial begin
    wait (KEY1 == 1'b0);
    wait (uart_count >= 4 &&
          uart_word == EXPECTED_UART_WORD &&
          seen_expected_sevenseg == 1'b1);
    $display("PASS: post-implementation result "
             "uart=%h sevenseg=%h time=%0t",
             uart_word, {SEL, leds}, $time);
    $finish;
end
```

Listing 9: 实现后时序仿真的参数及通过条件

实现后时序仿真的整体波形如图 9 所示。UART 输入约在 127.720 μs 完成，随后处理器输出结果并进入数码管扫描循环。稳定状态下，`SEL` 依次取 `1`、`2`、`4`、`8`，`leds` 对应取 `06`、`3F`、`3F`、`3F`，即组合值依次为 `106`、`23F`、`43F`、`83F`。位选切换附近出现的短暂中间值来自实现后不同布线路径的传播延迟，不影响各扫描状态的稳定值。

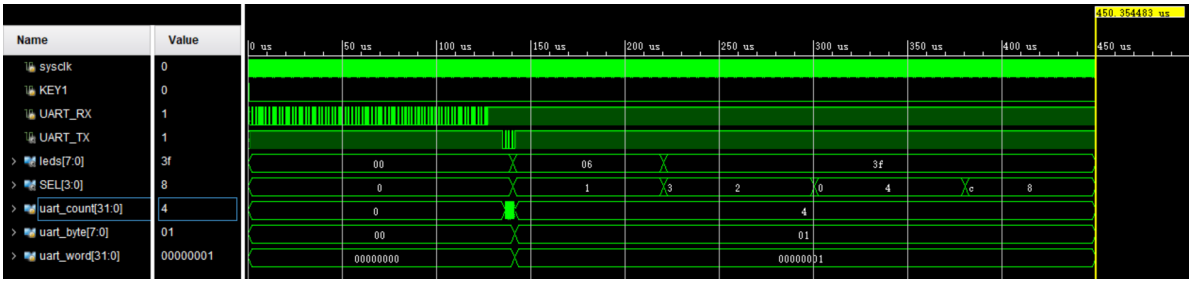


图 9: 实现后时序仿真的完整板级波形

UART 输出的局部波形如图 10 所示。四个字节分别在约 136.797 μs、138.507 μs、140.217 μs 和 141.917 μs 完成解码，`uart_count` 最终为 4，`uart_word` 最终为 `0x00000001`，与 RTL 功能仿真结果一致。

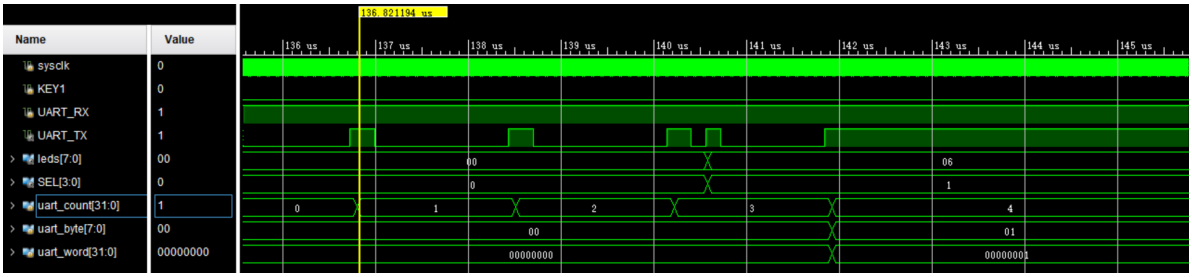


图 10: 实现后时序仿真的 UART 输出局部波形

Tcl Console 结果如图 11 所示。testbench 检测到 UART 返回 `00 00 00 01`，并在数码管出现 `83F` 后，于 380.909 μs 输出 `PASS`。前仿结束时间为 380.505 μs，二者十分接近；其中差异包含复位激励安排以及实现后传播延迟。该结果表明布局布线后的网表仍保持了正确的整机功能。

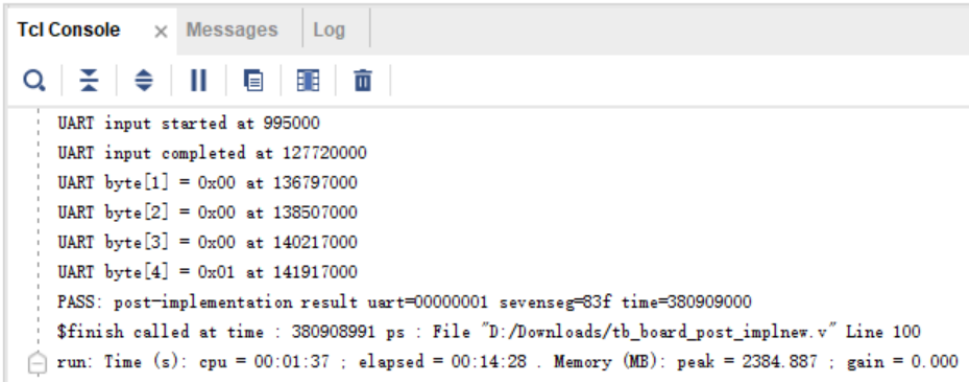


图 11: 实现后时序仿真的自动检查结果

6　综合情况

6.1　综合与实现结果

以实际上板使用的 `BoardTop` 为综合顶层，在 Vivado 中完成综合、布局与布线。综合后的顶层电路结构如图 12 所示。

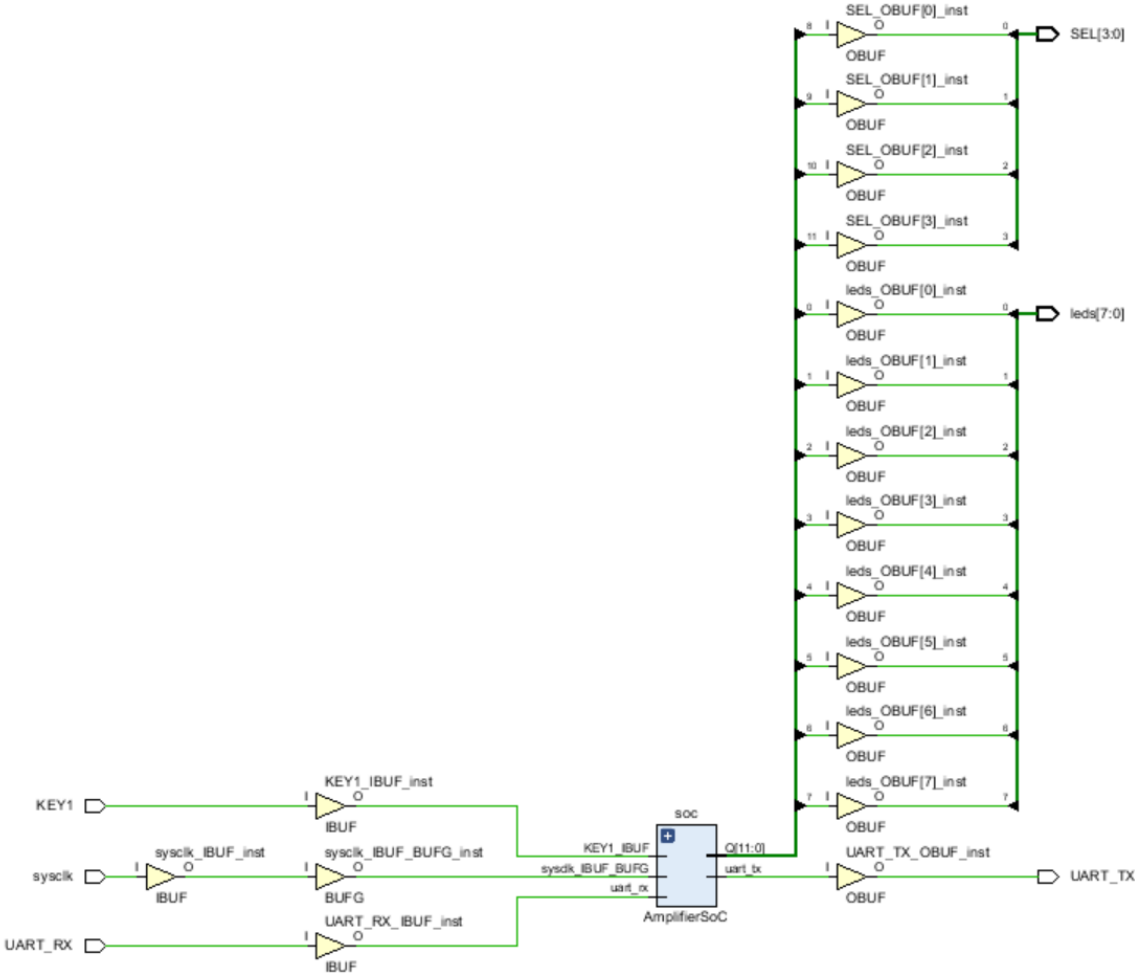


图 12: 综合后的顶层电路原理图

板级时钟 `sysclk` 经过输入缓冲器和全局时钟缓冲器 BUFG 后送入 `AmplifierSoC`；复位和 UART 接收信号经输入缓冲器接入 SoC，UART 发送、数码管段选和位选信号则通过输出缓冲器连接至 FPGA 引脚。综合后的连接关系与前述总体结构一致。

6.2　资源占用情况

实现后的总体资源利用率如图 13 所示。设计共使用 4349 个 LUT，占器件可用 LUT 的 20.91%；其中 2048 个 LUT 被用作 LUTRAM，占可用 LUTRAM 的 21.33%。此外，设计使用 1680 个触发器和 16 个 I/O，利用率分别为 4.04% 和 6.40%，总体资源占用处于较低水平。

Resource	Utilization	Available	Utilization %
LUT	4349	20800	20.91
LUTRAM	2048	9600	21.33
FF	1680	41600	4.04
IO	16	250	6.40

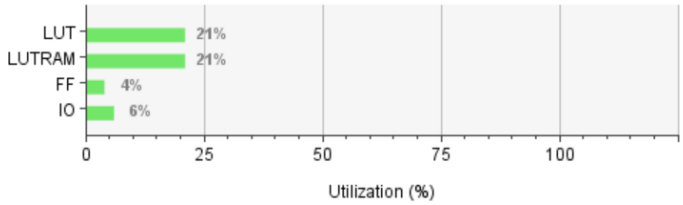


图 13: 实现后的总体资源利用率

各层次模块的资源占用如图 14 所示。流水线 CPU 使用 2052 个 LUT 和 1553 个触发器，其中寄存器堆使用 781 个 LUT 和 992 个触发器；数据 RAM 使用 2176 个 LUT，其中 2048 个被配置为存储单元。UART 外设使用 120 个 LUT 和 115 个触发器，七段数码管外设仅使用 12 个触发器。由此可见，主要资源消耗来自 CPU 数据通路、寄存器堆和采用 LUTRAM 实现的数据存储器，UART 与数码管外设的额外开销较小。

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	LUT Flip Flop Pairs (20800)	Bonded IOB (250)	BUFGCTRL (32)
BoardTop	4349	1680	1222	579	1439	2301	2048	437	16	1
soc (AmplifierSoC)	4349	1680	1222	579	1439	2301	2048	437	0	0
cpu (PipelineCPU)	2052	1553	158	64	808	2052	0	349	0	0
register_file (Re...	781	992	128	64	448	781	0	0	0	0
data_ram (DataRAM)	2176	0	1064	515	595	128	2048	0	0	0
sevenseg_peripher...	0	12	0	0	4	0	0	0	0	0
uart_peripheral (Ua...	120	115	0	0	63	120	0	87	0	0
uart_rx_unit (Uart...	65	48	0	0	32	65	0	46	0	0
uart_tx_unit (Uart...	55	48	0	0	30	55	0	40	0	0

图 14: 各层次模块的资源占用

6.3 时序性能与最高工作频率

对系统时钟施加 10.000 ns 的周期约束，对应目标频率为 100 MHz，如图 15 所示。

Name	Waveform	Period (ns)	Frequency (MHz)
sysclk	{0.000 5.000}	10.000	100.000

图 15: 系统时钟约束

实现后的时序摘要如图 16 所示。建立时间分析得到 WNS 为 0.230 ns、TNS 为 0；保持时间分析得到 WHS 为 0.072 ns、THS 为 0。建立时间、保持时间及脉冲宽度检查均不存在失败端点，说明设计满足 100 MHz 时钟约束。

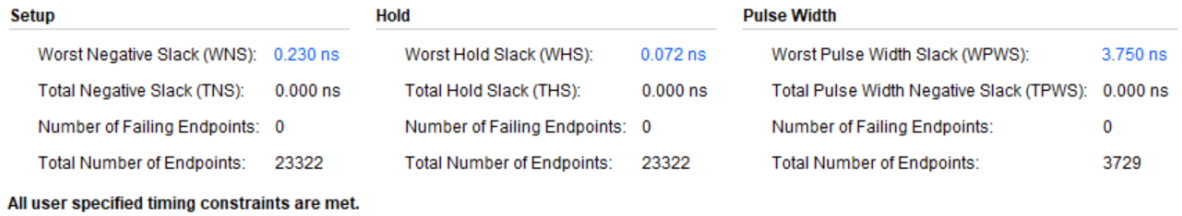


图 16: 实现后的时序摘要

根据当前布局布线结果和建立时间裕量，可近似估算处理器的最小时钟周期为

$$T_{\min} \approx T_{\text{constraint}} - \text{WNS} = 10.000\,\text{ns} - 0.230\,\text{ns} = 9.770\,\text{ns}.$$

因此，处理器的最高工作频率约为

$$F_{\max} \approx \frac{1}{T_{\min}} = \frac{1}{9.770\,\text{ns}} \approx 102.35\,\text{MHz}.$$

该数值是依据当前实现结果得到的近似估计；实际工程采用 100 MHz 板载时钟运行，并具有正的建立和保持时间裕量。

6.4 CPI 及每秒执行指令数

为避免 UART 波特率、串口轮询以及数码管延时程序影响统计结果，下面仅统计 UART 输入结束后节点放大算法的执行过程，即从 `PC=0x00000008` 处的第一条算法指令开始取指，到 `PC=0x00000104` 处的 `sw` 指令将结果写入 `0x1B04` 为止。

使用 MARS 的 Instruction Counter 统计指令数的方法不能直接用于本工程，因为实际在处理器上运行的 `amplifier_fpga.asm` 是面向裸机环境编写的程序，通过读取 `UART_CON` 和 `UART_RXD` 完成输入，并通过 `UART_TXD` 输出结果。MARS 并不具备相应的 UART 状态寄存器和收发逻辑，程序会停留在状态轮询循环中，因而无法复现实际执行路径。虽然原始 MARS 版汇编可以利用文件 syscall 完成输入输出，但其输入初始化、系统调用和输出程序均与 FPGA 版不同，由此得到的动态指令数不能代表处理器实际执行的指令数。

因此，在原有 SoC 功能仿真文件 `tb_amplifier_soc.v` 的基础上建立计数副本 `tb_amplifier_soc_cpi.v`，保留 UART 激励、结果检查和 PASS 判定，并增加周期计数器、退休指令计数器及 CPI 计算。关键实现如下：（本部分代码由 AI 辅助完成）

```
integer cpi_cycles;
integer cpi_instructions;
reg uart_receive_entered;
reg cpi_cycle_active;
reg cpi_retire_active;
reg cpi_finished;
real cpi_value;

```

```
always @(posedge clk) begin
    #1;    // Wait for pipeline registers to be updated.
    if (reset) begin
        cpi_cycles           = 0;
        cpi_instructions     = 0;
        uart_receive_entered = 1'b0;
        cpi_cycle_active     = 1'b0;
        cpi_retire_active    = 1'b0;
        cpi_finished         = 1'b0;
    end
    else if (!cpi_finished) begin   // UART receive routine entry.
        if (debug_pc == 32'h000001a0) uart_receive_entered = 1'b1;

        // Start when the real PC=0x8 enters IF.
        if (!cpi_cycle_active && uart_receive_entered &&
            debug_pc == 32'h00000008) begin
            cpi_cycle_active = 1'b1;
            cpi_cycles = 0;
        end
        else if (cpi_cycle_active) begin
            cpi_cycles = cpi_cycles + 1;
        end

        // Start retirement counting when PC=0x8 reaches WB.
        if (!cpi_retire_active &&
            dut.cpu.debug_wb_pc == 32'h00000008)
            cpi_retire_active = 1'b1;

        // A bubble produced by stall or flush carries PC=0.
        if (cpi_retire_active &&
            dut.cpu.debug_wb_pc != 32'h00000000)
            cpi_instructions = cpi_instructions + 1;

        // The store at PC=0x104 writes the final result.
        if (cpi_retire_active &&
            dut.cpu.debug_wb_pc == 32'h00000104) begin
            cpi_value = 1.0 * cpi_cycles / cpi_instructions;
            $display("CPI_RESULT: cycles=%0d instructions=%0d CPI=%0.4f",
                     cpi_cycles, cpi_instructions, cpi_value);
            cpi_finished = 1'b1;
        end
    end
end
```

Listing 10: CPI 动态统计关键代码

程序开始执行时，`jal uart_receive_input` 尚未在 ID 阶段完成跳转，IF 阶段会短暂顺序取到一次 `PC=0x8`，随后该错误路径指令被 `flush`。为避免将其误认为算法入口，计数器首先检测 UART 接收函数入口 `PC=0x1A0`，只有在进入该函数后再次出现 `PC=0x8`，才启动周期统计。

周期数从第一条算法指令进入 IF 时开始，每个时钟周期加 1。指令数则在该指令到达 WB 后开始统计，从而避免计入流水线中尚未排空的 UART 接收指令。当前处理器会将 stall 和 `flush` 产生的气泡 PC 置零，因此可使用 `debug_wb_pc != 0` 判断真实完成的指令。该方法同时统计算术、访存、分支和跳转指令，而不是仅统计写寄存器的指令。代码中的 `#1` 用于等待非阻塞赋值更新完成，仅影响仿真，不参与硬件综合。

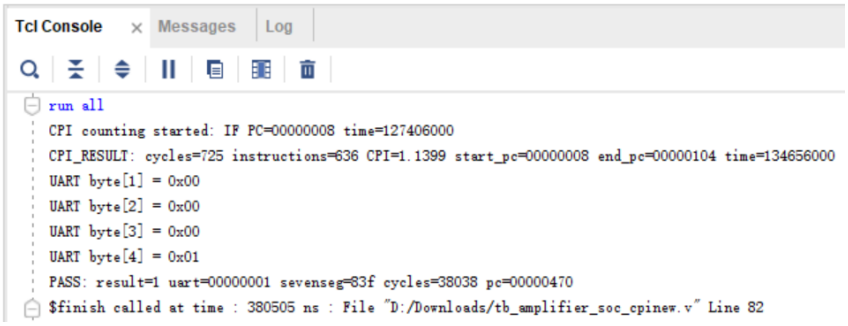


图 17: CPI 统计的 Tcl Console 输出

统计起止时刻分别为 $t_{\text{start}} = 127406000\,\text{ps}$，$t_{\text{end}} = 134656000\,\text{ps}$。由于仿真时钟周期为 $10\,\text{ns}$，故总周期数为

$$C = \frac{134656000 - 127406000}{10000} = 725.$$

同一区间内实际完成的指令数为 $N = 636$，因此

$$\text{CPI} = \frac{725}{636} \approx 1.1399.$$

平均每秒执行指令数可由处理器工作频率除以 CPI 得到：

$$R_{\text{inst}} = \frac{f_{\text{clk}}}{\text{CPI}}.$$

本工程未对板载 $100\,\text{MHz}$ 系统时钟进行分频，处理器实际工作频率为 $100\,\text{MHz}$，因此

$$R_{\text{inst}} = \frac{100\,\text{MHz}}{1.1399} \approx 87.72 \times 10^6 \text{ instruction/s}.$$

即在该组节点放大器测试数据下，处理器平均每秒约执行 87.72 百万条指令。

图中 `PASS cycles=38038` 是原 testbench 从复位结束一直统计到 UART 输出和数码管显示均正确的周期数，包含串口输入输出及显示延时，不属于 CPI 的统计区间。

此外，CPI 也可直接通过统计计算得到，可对仿真所得结果进行印证，详见附录 B。

7 硬件调试情况

在调试过程中，我们先分别验证了流水线 CPU、UART 和存储器等模块，再进行完整 SoC 仿真，最后完成综合、实现与上板测试。调试过程中遇到的主要问题如下：

1. 调试初期，整机仿真出现超时，PC 持续递增但程序没有产生任何结果。查看 Instruction ROM 后发现其中均为 NOP，原因是 Vivado 的仿真工作目录与预期不一致，导致 `$readmemh` 未能通过相对路径找到机器码文件。随后将 ROM、RAM 初始化文件改为明确的工程路径，并在 testbench 开始时检查 ROM 和 RAM 的首个数据，这样在仿真开始阶段就可以及时发现加载错误。

2. 流水线各模块连接完成后，曾出现 ID/EX 流水寄存器控制方式的综合警告。原有代码没有清晰地区分异步复位与 branch、stall 产生的同步气泡。修改后仅将 reset 作为异步控制，而把 stall 和 `flush` 统一放在时钟上升沿处理，使流水寄存器的更新逻辑更加明确，并消除了相应的 critical warning。

3. 工程最初在 100 MHz 时钟约束下出现较大的负时序裕量。检查关键路径后发现，虽然节点放大器程序并未使用乘法指令，但 ALU 中保留的组合乘法仍可能被综合为较长的数据通路。由于该功能不属于本程序实际需要的指令子集，最终通过参数关闭组合乘法逻辑，并重新整理流水寄存器控制，成功使 WNS 转为正值。

4. 上板初期还遇到了数码管显示为 `1000` 而不是 `0001` 的小问题，主要原因是我对数码管位选顺序没有记清楚。重新核对实验板原理图并调整 XDC 中 `SEL[0]` 至 `SEL[3]` 的管脚对应关系后，结果恢复正常。

5. UART 提高版调试时，小规模样例能够返回结果，而完整样例曾出现长时间无响应。检查串口助手底部的 Byte 数后发现，理论上应发送的 18 个 32 位整数共为 72 字节，实际发送流中混入了换行等额外字符，破坏了每四字节组成一个大端整数的边界。改用纯 Hex 模式并严格检查发送字节数后，处理器能够正确接收数据、完成计算，并返回正确结果 `00 00 00 01`。

8 思想体会

五级流水线 CPU 大作业涉及处理器数据通路、控制逻辑、汇编程序、存储器映射、UART 外设和 FPGA 时序等多个方面，整体难度很高。这让我认识到，完成工程时，一定要在动手前先想好整体的架构，再将其分解为流水寄存器、控制、冒险处理、存储器和外设等模块逐一实现，最后通过模块仿真、整机仿真和上板测试逐层验证。

本次实验使我进一步理解复习了流水线等数据通路，以及转发、stall 和 `flush` 的工程实现。通过汇编程序移植、Verilog 编写、Vivado 时序分析以及多层次调试，我在系统设计、问题定位和软硬件协同设计等方面均得到了较大提高。同时，本次实验也说明，对于规模较大的工程，清晰的整体规划、明确的模块接口和循序渐进的验证方法，往往比在整机出现问题后反复修改更为重要。

最后，再次感谢老师和助教们的辛苦付出！

A　优化设计说明

A.1　优化目标与总体方法

优化目标是在不使用 CPU 时钟分频模块的条件下，使五级流水线处理器能直接由实验板 100 MHz 系统时钟驱动，在完成 UART、数据存储器和七段数码管等模块集成后仍满足时序要求，且留出尽可能多的时序裕量。

优化时不通过插入额外等待周期降低组合逻辑压力，以免牺牲 CPI，而主要从关键路径定位、无效组合逻辑裁剪、流水寄存器控制方式和寄存器堆时序结构等方面调整。

优化时，首先通过功能仿真确认功能正确；随后在 10 ns 时钟约束下综合和实现，根据 Timing Summary 中的 WNS、TNS 以及最差路径定位时序瓶颈；完成 RTL 修改后重新进行前仿、综合和实现，并最终通过后仿与上板测试验证优化没有破坏原功能。

工程顶层为 `BoardTop`。板载时钟 `sysclk` 不分频，直接连接至 `AmplifierSoC`：

```
module BoardTop(
    input        sysclk,
    input        KEY1,
    input        UART_RX,
    output       UART_TX,
    output [7:0] leds,
    output [3:0] SEL
);

    AmplifierSoC soc(
        .clk     (sysclk),
        .reset   (KEY1),
        .uart_rx (UART_RX),
        .uart_tx (UART_TX),
        .seg     (leds),
        .an      (SEL)
    );

endmodule
```

Listing 11: 板载时钟直接驱动 SoC

约束文件中将 `sysclk` 的周期设置为 10.000 ns，即按照 100 MHz 进行综合和布线，而不是先降低 CPU 时钟频率再进行时序分析。

```
set_property -dict {PACKAGE_PIN R4 IOSTANDARD LVCMOS33} \
    [get_ports {sysclk}]
create_clock -name sysclk -period 10.000 \
    [get_ports {sysclk}]
```

Listing 12: 100 MHz 系统时钟约束

A.2 初始时序问题与关键路径分析

基础流水线初版在 10.000 ns 时钟约束下得到的主要时序指标为

$$\mathrm{WNS} = -4.516\,\mathrm{ns}, \qquad \mathrm{TNS} = -170.234\,\mathrm{ns}.$$

其中，WNS 为负，表示最差建立时间路径未满足约束。

由约束周期与 WNS 可以估算该关键路径所需时间约为

$$T_{\mathrm{path}} \approx T_{\mathrm{constraint}} - \mathrm{WNS} = 10.000\,\mathrm{ns} - (-4.516\,\mathrm{ns}) = 14.516\,\mathrm{ns}.$$

因此该版本虽然能够通过功能仿真，但不能认为可以稳定工作在 100 MHz。

进一步检查实现后的关键路径和 ALU 结构后发现，EX 阶段存在一个 32 位组合乘法运算 `in1 * in2`。由于 ALU 的运算类型由运行时控制信号 `alu_ctl` 选择，即使当前节点放大器程序中没有执行 `mul` 指令，综合器也不能仅根据软件运行情况自动忽略这一分支，因而仍可能实现完整的乘法器以及乘法结果选择逻辑。

组合乘法器的逻辑级数和布线延迟明显大于加法、逻辑运算及比较运算。当其输出进入 ALU 结果多路选择器后，会对整个 EX 阶段的寄存器到寄存器路径产生影响，因此首先对该无实际用途的长组合路径进行处理。

A.3 关键 RTL 优化措施

A.3.1 参数化裁剪组合乘法器

该汇编程序使用加法、减法、比较、移位、访存和控制转移等指令完成算法，并不需要乘法指令。因此，在 ALU 中增加参数 `ENABLE_MUL`，默认值设置为 0；在 `PipelineCPU` 中实例化 ALU 时也显式将该参数固定为 0。

```
module ALU #(
    parameter ENABLE_MUL = 1'b0
)(
    input  [31:0] in1,
    input  [31:0] in2,
    input  [4:0]  alu_ctl,
    output reg [31:0] out,
    output        zero
);

    always @(*) begin
        case (alu_ctl)
            ALU_AND:  out = in1 & in2;
            ALU_OR:   out = in1 | in2;
            ALU_ADD:  out = in1 + in2;
            ALU_SUB:  out = in1 - in2;
```

```
            ALU_SLT:  out = ($signed(in1) < $signed(in2)) ? 32'd1 : 32'd0;
            ALU_SLL:  out = in2 << in1[4:0];
            ALU_SRL:  out = in2 >> in1[4:0];
            ALU_SRA:  out = $signed(in2) >>> in1[4:0];
            ALU_MUL:  out = ENABLE_MUL ? (in1 * in2) : 32'b0;
            default:  out = 32'b0;
        endcase
    end

    assign zero = (out == 32'b0);

endmodule
```

Listing 13: ALU 中乘法运算的参数化控制

PipelineCPU 中的实例化方式如下。

```
ALU #(
    .ENABLE_MUL(1'b0)
) alu (
    .in1     (ex_alu_in1),
    .in2     (ex_alu_in2),
    .alu_ctl (id_ex_alu_ctl),
    .out     (ex_alu_result),
    .zero    (ex_alu_zero)
);
```

Listing 14: 在流水线处理器中关闭组合乘法器

由于 ENABLE_MUL 是综合时可以确定的常量，Vivado 能够通过常量传播将乘法分支化简为常数 0，并删除未被使用的乘法器资源。与直接保留 in1 * in2 相比，这种方式缩短了 ALU 组合路径，同时保留了参数化接口，便于后续修改。

该优化以实际运行程序所需的指令集合为依据，不会影响节点放大器程序的执行结果。不过，它也意味着当前可烧录版本其实并未真正实现 mul 指令：如果程序执行该指令，ALU 将返回 0。因此，指令支持范围不应包含硬件乘法。若后续需要恢复乘法功能，应重新启用该参数，并重新完成时序收敛，而不能直接恢复原组合乘法后沿用当前时序结论。

A.3.2 整理流水寄存器的同步控制逻辑

调试过程中，Vivado 曾对 ID/EX 流水寄存器的控制方式给出 critical warning。原有写法没有清晰区分复位、分支 flush 和 stall 气泡，使综合器可能将不同性质的控制条件解释为混合的同步和异步控制。

修改后，只有 reset 出现在异步控制分支中；分支 flush 和 stall 均在时钟上升沿同步写入气泡。正常情况下，译码结果在上升沿写入 ID/EX 流水寄存器。

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        id_ex_pc             <= 32'b0;
        id_ex_rs_data        <= 32'b0;
        id_ex_rt_data        <= 32'b0;
        id_ex_write_reg      <= 5'b0;
        id_ex_reg_write      <= 1'b0;
        id_ex_mem_read       <= 1'b0;
        id_ex_mem_write      <= 1'b0;
        id_ex_branch_type    <= BR_NONE;
    end else if (ex_branch_taken || stall) begin
        // 同步写入气泡
        id_ex_pc             <= 32'b0;
        id_ex_rs_data        <= 32'b0;
        id_ex_rt_data        <= 32'b0;
        id_ex_write_reg      <= 5'b0;
        id_ex_reg_write      <= 1'b0;
        id_ex_mem_read       <= 1'b0;
        id_ex_mem_write      <= 1'b0;
        id_ex_branch_type    <= BR_NONE;
    end else begin
        // 正常锁存译码结果
        id_ex_pc             <= if_id_pc;
        id_ex_rs_data        <= id_rs_data;
        id_ex_rt_data        <= id_rt_data;
        id_ex_write_reg      <= id_write_reg;
        id_ex_reg_write      <= id_reg_write;
        id_ex_mem_read       <= id_mem_read;
        id_ex_mem_write      <= id_mem_write;
        id_ex_branch_type    <= id_branch_type;
    end
end
```

Listing 15: ID/EX 流水寄存器的同步更新

IF/ID 流水寄存器也采用明确的同步更新优先级。分支或跳转成立时写入 NOP，stall 时不执行更新，从而保持原指令和 PC；不存在冒险时才锁存新的取指结果。

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        if_id_pc    <= 32'b0;
        if_id_pc4   <= 32'b0;
        if_id_instr <= NOP;
    end else if (ex_branch_taken || id_jump_take) begin
        if_id_pc    <= 32'b0;
```

```
        if_id_pc4   <= 32'b0;
        if_id_instr <= NOP;
    end else if (!stall) begin
        if_id_pc    <= pc;
        if_id_pc4   <= pc4;
        if_id_instr <= instr_rdata;
    end
end
```

Listing 16: IF/ID 流水寄存器的保持与 flush

这种修改的主要作用是消除混合同步、异步控制的综合警告，使 stall 和 flush 通过流水寄存器的正常数据输入端实现，并使寄存器更新语义与五级流水线的时序关系保持一致。与乘法器裁剪相比，该修改不能单独对应某一项 WNS 改善数值，但它提高了综合结果的确定性和工程可靠性，是时序收敛前必须完成的结构整理。

A.3.3 避免下降沿写寄存器造成半周期路径

若采用上升沿更新流水寄存器、下降沿写寄存器堆的方法解决 WB 与 ID 的同周期数据冒险，则部分路径只有半个时钟周期可以完成。在 100 MHz 时钟下，半周期仅为 5 ns，容易形成更大的额外时序压力。

本设计的寄存器堆统一在上升沿写入，读端口保持组合读取；当本周期写回寄存器与 ID 阶段读取寄存器相同时，通过组合旁路直接返回写回数据。

```
assign read_data1 =
    (read_reg1 == 5'd0) ? 32'b0 :
    ((reg_write &&
      write_reg == read_reg1 &&
      write_reg != 5'd0) ? write_data : regs[read_reg1]);

assign read_data2 =
    (read_reg2 == 5'd0) ? 32'b0 :
    ((reg_write &&
      write_reg == read_reg2 &&
      write_reg != 5'd0) ? write_data : regs[read_reg2]);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 1; i < 32; i = i + 1)
            regs[i] <= 32'b0;
    end else if (reg_write && write_reg != 5'd0) begin
        regs[write_reg] <= write_data;
    end
end
```

Listing 17: 寄存器堆的上升沿写入与同周期旁路

该结构在保证 WB 阶段写回值能被 ID 阶段立即读取的同时，使设计中的主要状态单元统一在同一时钟边沿工作，避免人为产生半周期寄存器路径，也简化了时钟结构。

A.3.4 限制无效的转发路径

EX 阶段的两个 ALU 操作数可以从 ID/EX、EX/MEM 或 MEM/WB 三级数据中选择。对于普通 ALU 指令，EX/MEM 中已经存在可用结果，可以直接转发；但对于 `lw`，EX/MEM 阶段只有访存地址，真正的加载数据尚未从数据存储器返回，不能作为有效的转发数据。

因此，处理器通过 `ex_mem_can_forward` 排除加载指令在 EX/MEM 阶段产生的无效转发，并在 load-use 情况下由冒险检测模块插入一个 stall，随后从 MEM/WB 转发实际加载结果。

```
assign ex_mem_forward_data =
    (ex_mem_result_src == RES_LINK)
        ? ex_mem_link_value
        : ex_mem_alu_result;

assign ex_mem_can_forward =
    (ex_mem_result_src != RES_MEM);

ForwardingUnit forwarding_unit(
    .id_ex_rs           (id_ex_rs),
    .id_ex_rt           (id_ex_rt),
    .ex_mem_reg_write   (ex_mem_reg_write),
    .ex_mem_can_forward (ex_mem_can_forward),
    .ex_mem_write_reg   (ex_mem_write_reg),
    .mem_wb_reg_write   (mem_wb_reg_write),
    .mem_wb_write_reg   (mem_wb_write_reg),
    .forward_a          (forward_a),
    .forward_b          (forward_b)
);
```

Listing 18: 对 EX/MEM 转发来源进行限制

这一处理首先保证了流水线功能正确，同时使 EX 阶段只在数据已经有效时才选择 EX/MEM 转发结果，避免将尚未完成的存储器读取路径错误接入 ALU。普通数据相关仍通过转发解决，不额外增加 stall，从而在时序优化同时保持流水线的执行效率。

A.4 优化后的时序结果

完成组合乘法器裁剪和流水寄存器控制逻辑整理后，基础版处理器在相同的 10 ns 时钟约束下得到

$$\mathrm{WNS} = 0.173\,\mathrm{ns}, \qquad \mathrm{TNS} = 0,$$

原有建立时间违例已经消除。

随后在此基础上加入 UART 接收、UART 发送和相应的存储器映射外设，形成最终的 UART Bonus 工程。最终工程以 `BoardTop` 为综合和实现顶层，在 100 MHz 约束下得到

$$\mathrm{WNS} = 0.230\,\mathrm{ns}, \qquad \mathrm{TNS} = 0,$$

以及

$$\mathrm{WHS} = 0.072\,\mathrm{ns}, \qquad \mathrm{THS} = 0.$$

WNS 和 WHS 均为正，TNS 和 THS 均为 0，说明实现后的所有建立时间和保持时间检查均通过。

根据最终实现的 WNS，对最小时钟周期进行近似折算：

$$T_{\min} \approx T_{\mathrm{constraint}} - \mathrm{WNS} = 10.000\,\mathrm{ns} - 0.230\,\mathrm{ns} = 9.770\,\mathrm{ns}.$$

由此得到估算的最高时钟频率为

$$F_{\max} \approx \frac{1}{T_{\min}} = \frac{1}{9.770\,\mathrm{ns}} \approx 102.35\,\mathrm{MHz}.$$

该数值是根据 10.000 ns 约束下的实现后 slack 折算得到的时序上限；实验板上的实际运行频率仍为板载时钟提供的 100 MHz。因此，本工程能够在不进行时钟分频的情况下满足 100 MHz 统一测试条件。

A.5 补充说明

时序优化不能仅依据 WNS 由负转正判断完成，还必须确认修改没有改变处理器的功能。因此，每一次进行优化设计之后，还依次进行了前仿、后仿、综合上板等步骤来验证功能依然正常。

综上所述，本设计的主要时序瓶颈是实际程序未使用但仍被综合实现的 32 位组合乘法器。通过参数化关闭乘法功能，Vivado 可以裁剪该长组合路径；同时，通过整理流水寄存器的 reset、flush 和 stall 控制方式，采用统一上升沿写入的寄存器堆及同周期旁路，并限制无效的加载转发路径，使整个流水线的时序关系更加明确。

优化过程没有增加额外流水级或人为等待周期，因而没有为了提高时钟频率而额外降低程序执行效率。

当前结论仅适用于本实验所需的指令集合和目前的 RTL 配置。若后续重新启用 `mul`、扩大存储器规模、改变外设结构或修改流水级之间的组合逻辑等，都需要重新综合和时序分析，不能直接沿用报告里给出的最高频率。

B 对 CPI 计算结果的印证

为验证 testbench 输出的指令数、周期数和 CPI，可通过直接计算进行交叉检查。首先，以相同的 18 个输入整数初始化数据存储器，按照 FPGA 版汇编的真实指令语义，从 `PC=0x8` 执行至 `PC=0x104`。独立重放得到动态执行指令数为 636，最终写入 `0x1B04` 的结果为 1，与 RTL 仿真完全一致。

若暂不考虑冒险，636 条指令从第一条进入 IF，到最后一条 `sw` 完成 MEM，需要 $636 + 3 = 639$ 个周期。根据实际动态执行路径，各类额外开销如表 5 所示。

表 5: 流水线额外周期复算

原因	动态次数	单次损失（周期）	增加周期数
`j`、`jal`、`jr`	31	1	31
成功跳转的条件分支	16	2	32
load-use stall	19	1	19
`jr` 等待跳转寄存器	–	–	4
合计	–	–	86

其中，跳转指令在 ID 阶段确定目标，需要取消一条 IF 阶段指令；成功跳转的条件分支在 EX 阶段确定，需要对 IF、ID 两级执行 `flush`；load-use 相关需要 stall 一个周期；部分 `jr` 还需等待跳转地址写回寄存器。由此复算得到

$$C = 639 + 31 + 32 + 19 + 4 = 725,$$

与 testbench 的周期统计完全相同。最终仍有

$$\mathrm{CPI} = \frac{725}{636} = 1.1399,$$

说明仿真输出的指令数、周期数和 CPI 可以相互验证。